

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

CHATBOT NA VYTVÁRANIE KONTROLNÝCH
ZOZNAMOV NA TESTOVANIE SOFTVÉRU
BAKALÁRSKA PRÁCA

2026
MATÚŠ ŠLECHTA

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

CHATBOT NA VYTVÁRANIE KONTROLNÝCH ZOZNAMOV NA TESTOVANIE SOFTVÉRU

BAKALÁRSKA PRÁCA

Študijný program: Aplikovaná informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra aplikovanej informatiky
Školiteľ: doc. Mgr. Yevheniia Kataieva, PhD.

Bratislava, 2026
Matúš Šlechta



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Matúš Šlechta
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Chatbot na vytváranie kontrolných zoznamov na testovanie softvéru
Chatbot for creating checklists for software testing

Anotácia: Cieľom bakalárskej práce je návrh a implementácia chatbota, ktorý podporuje proces testovania softvéru prostredníctvom automatizovaného vytvárania kontrolných zoznamov (checklistov).

Vedúci: doc. Mgr. Yevheniia Kataieva, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 06.10.2025

Dátum schválenia: 09.10.2025
doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

PodĎakovanie: Rád by som sa poďakoval svojej školiteľke doc. Mgr. Yevheniia Kataieva, PhD. za odborné vedenie, cenné pripomienky a čas, ktorý mi venoval pri konzultáciách počas celého obdobia prípravy tejto bakalárskej práce.

Abstrakt

Bakalárska práca sa zaoberá návrhom a implementáciou konverzačného systému, ktorý na základe textového opisu projektu generuje testovací checklist. Cieľovou skupinou systému sú začínajúci tester, malé vývojové tímy a študenti, ktorí potrebujú zostaviť štruktúrovaný testovací plán bez detailnej znalosti testovacích metodík.

Systém je postavený na hybridnej architektúre, ktorá kombinuje statických špecialistov pre štyri najčastejšie typy aplikácií (web, mobile, desktop, API) s dynamickým generovaním špecialistu pre projekty mimo týchto kategórií. Pri dynamickom generovaní systém počas behu vytvorí novú schému, nápovedy k jednotlivým poliam a doménové pravidlá pre vedenie konverzácie. Konverzácia s používateľom prebieha v prirodzenom jazyku a využíva lokálny jazykový model spúšťaný cez Ollama. Výsledný checklist je rozdelený do troch prioritných úrovní a dá sa exportovať do XLSX súboru s farebným odlíšením priorít.

Funkčnosť systému som overil na dvoch typoch projektov, ktoré reprezentujú zabudovaného aj dynamicky vygenerovaného špecialistu. Pri overovaní som zaznamenal silné stránky aj obmedzenia, najmä závislosť kvality výstupu od formulácie úvodnej správy a od konkrétneho jazykového modelu.

Kľúčové slová: testovanie softvéru, testovací checklist, chatbot, jazykové modely, generovanie testov, Ollama

Abstract

This bachelor thesis describes the design and implementation of a conversational system that generates a testing checklist from a textual project description. The target audience consists of beginner testers, small development teams, and students who need to create a structured testing plan without detailed knowledge of testing methodologies.

The system is built on a hybrid architecture that combines static specialists for the four most common application types (web, mobile, desktop, API) with dynamic specialist generation for projects outside these categories. During dynamic generation, the system creates a new schema, hints for individual fields, and domain rules for guiding the conversation at runtime. The conversation runs in natural language and is powered by a local language model served through Ollama. The resulting checklist is divided into three priority levels and can be exported to an XLSX file with priority-based color coding.

The system was verified on two project types representing both a built-in and a dynamically generated specialist. The verification revealed strengths as well as limitations, in particular the dependency of output quality on the wording of the initial message and on the specific language model in use.

Keywords: software testing, testing checklist, chatbot, large language models, test generation, Ollama

Obsah

Úvod	1
1 Východiská práce	3
1.1 Význam testovania softvéru	3
1.2 Štatistika a náklady spojené s kvalitou softvéru	4
1.3 Význam testovacieho procesu	4
1.4 Charakteristiky kvality softvéru podľa ISO/IEC 25010	6
2 Analýza existujúcich softvérových riešení	9
2.1 Čo je chatbot	9
2.2 ChatGPT ako príklad moderného chatbota	10
2.3 GitHub Copilot	11
2.4 Slackbot AI	13
2.5 Porovnanie vybraných AI nástrojov	14
3 Analýza problému a špecifikácia riešenia	15
3.1 Identifikácia problému	15
3.2 Cieľ navrhovaného riešenia	15
3.3 Cieľová skupina používateľov	16
3.4 Prínosy navrhovaného riešenia	16
3.5 Funkčné a nefunkčné požiadavky	17
3.5.1 Funkčné požiadavky	17
3.5.2 Nefunkčné požiadavky	19
4 Návrh systému	21
4.1 Prehľad navrhovaného riešenia	21
4.2 Otázky a odpovede chatbota	21
4.2.1 Určenie typu aplikácie	21
4.2.2 Všeobecný kontext projektu	22
4.2.3 Podmienené otázky	22
4.2.4 Integrácie a externé služby	23

4.2.5	otázky špecifické pre API	24
4.3	Spracovanie odpovedí a udržiavanie kontextu	24
4.4	Generovanie checklistu	24
5	Implementácia riešenia	27
5.1	Architektúra	27
5.2	Priebeh spracovania požiadavky	35
5.2.1	Začiatok aplikácie	35
5.2.2	Vytvorenie novej relácie	35
5.2.3	Načítanie existujúcej relácie	36
5.2.4	Prvá správa a detekcia typu aplikácie	37
5.2.5	Cyklus dopĺňania schémy	38
5.2.6	Dynamické generovanie špecialistu	40
5.3	Diagram aktivity systému	41
6	Overenie a zhodnotenie riešenia	45
6.1	Spôsob overenia riešenia	45
6.2	Príklady použitia	46
6.2.1	Webová aplikácia: TaskFlow	46
6.2.2	CLI nástroj cez OtherAgent	49
6.3	Overenie s externým používateľom	53
6.3.1	Profil testera a priebeh sedenia	54
6.3.2	Pozorovania k používateľskému rozhraniu	54
6.3.3	Pozorovania ku komunikácii s chatbotom	54
6.3.4	Pozorovania k rýchlosti odozvy	55
6.3.5	Pozorovania k vygenerovanému checklistu	55
6.3.6	Súhrn pozorovaní z externého overenia	55
6.4	Screenshoty rozhrania	56
6.4.1	Prihlasovacie rozhranie	56
6.4.2	Hlavné okno po prihlásení	57
6.4.3	Zoznam konverzácií a stav po dokončení	57
6.4.4	Exportovaný checklist	58
6.5	Pozorovania k správaniu systému	59
6.6	Identifikované obmedzenia	61
6.6.1	Závislosť výstupu od formulácie úvodnej správy	61
6.6.2	Neúplné zachytenie polí z rozsiahlych odpovedí	61
6.6.3	Závislosť kvality od jazykového modelu	62
6.6.4	Statické rizikové pravidlá pri preddefinovaných kategóriách	62
	Záver	63

Príloha A: Zdrojový kód	71
Príloha B	73
Checklist projektu TaskFlow	74
Checklist projektu EshopBackend	76
Checklist projektu CLITool	77

Zoznam obrázkov

1.1	V-model v kontexte vývoja a testovania softvéru	5
2.1	Návrh kódu od GitHub Copilot [9]	12
5.1	Architektúra systému - vrstvomé usporiadanie komponentov a tok dát medzi nimi.	34
5.2	Sekvenčný diagram fázy Začiatok aplikácie.	35
5.3	Sekvenčný diagram fázy Vytvorenie novej relácie.	36
5.4	Sekvenčný diagram fázy Načítanie existujúcej relácie.	36
5.5	Sekvenčný diagram fázy Prvá správa a detekcia typu aplikácie.	38
5.6	Sekvenčný diagram fázy Cyklus dopĺňania schémy.	39
5.7	Sekvenčný diagram fázy Dynamické generovanie špecialistu.	41
5.8	Diagram Aktivít.	42
6.1	Skrátený prepis konverzácie pre projekt TaskFlow.	47
6.2	Skrátený úsek konverzácie pre projekt CLITool.	50
6.3	Prihlasovací formulár.	56
6.4	Hlavné okno aplikácie po prihlásení a vytvorení nového účtu, ešte bez konverzácií.	57
6.5	Stav aplikácie po dokončení konverzácie pre projekt CLITool.	58
6.6	Exportovaný checklist v podobe XLSX súboru.	59
6.7	Checklist projektu TaskFlow — sekcia HIGH PRIORITY	74
6.8	Checklist projektu TaskFlow — sekcie MEDIUM a LOW PRIORITY	75
6.9	Checklist projektu EshopBackend	76
6.10	Checklist projektu CLITool — sekcia HIGH PRIORITY.	77

Zoznam tabuliek

1.1	Zjednodušený prehľad fáz testovacieho procesu podľa ISTQB a SWE-BOK [12, 4]	6
2.1	Porovnanie vybraných AI nástrojov	14
6.1	Rozklad odpovede používateľa o autentifikácii na jednotlivé polia schémy.	51

Úvod

Vývoj softvéru sa za posledné roky zrýchlil. Nástroje založené na veľkých jazykových modeloch, ako GitHub Copilot a ChatGPT, pomáhajú vývojárom písať kód rýchlejšie a hľadať riešenia bez prerušenia práce. Do produkčných verzií sa tak dostáva viac kódu za kratší čas. Testovanie musí stíhať túto rýchlosť a zároveň pokrývať systémy, ktoré sú čoraz zložitejšie.

Najťažšie sa to znáša v menších vývojových tímoch a u začínajúcich testerov, ktorým chýbajú časové aj znalostné zdroje pre dôkladnú prípravu testovacieho plánu. Príprava plánu si tradične vyžaduje skúseného testera, ktorý vie z opisu projektu identifikovať rizikové oblasti a určiť priority. Pre menej skúseného človeka je to úloha náročná, lebo si vyžaduje znalosť testovacích metodík (ISTQB, OWASP, modely kvality podľa noriem) aj prehľad o tom, aké riziká bývajú spojené s konkrétnym typom aplikácie. Takéto tímy preto buď tento krok preskakujú, alebo používajú generické šablóny, ktoré obsahujú všetko pre všetky typy aplikácií, a preto sú ťažko použiteľné v konkrétnom projekte.

Generatívne jazykové modely môžu túto medzeru zaplniť. Existujúce nástroje ako Copilot či ChatGPT vedia generovať aj testovací obsah, ale pracujú reaktívne, teda odpovedajú na otázku, ktorú im používateľ položí. Začínajúci tester však nevie, akú otázku má položiť. To sa dá riešiť konverzačným systémom, ktorý sa pýta sám a pomáha používateľovi odhaliť oblasti, na ktoré by inak zabudol.

Predmetom tejto bakalárskej práce je návrh a implementácia konverzačného systému, ktorý na základe textového opisu projektu vygeneruje testovací checklist. Systém má poslúžiť ako podporný nástroj pre začínajúcich testerov, malé vývojové tímy a študentov, ktorí potrebujú rýchlo zostaviť testovací plán pre svoj projekt, ale nemajú časový ani znalostný priestor pre detailnú analýzu rizík. Konverzácia s používateľom prebieha v prirodzenom jazyku, systém z odpovedí extrahuje informácie o type aplikácie, jej funkcionalitách, integráciách a požiadavkách, a na ich základe zostaví štruktúrovaný checklist rozdelený do troch prioritných úrovní.

Kapitola 1

Východiská práce

1.1 Význam testovania softvéru

Testovanie patrí medzi základné aktivity vývoja softvérového produktu [24]. Jeho cieľom je odhaľovať chyby, zvyšovať spoľahlivosť systému a overiť, či systém správne reaguje na očakávané aj neočakávané vstupy. Bez systematického testovania nie je možné dostatočne overiť, či softvér poskytuje očakávané správanie pri relevantných testovacích prípadoch [4].

Testovanie neprebíha len v záverečných fázach vývoja. Prístupy ako testovaním riadený vývoj (TDD) [2, 21] alebo kontinuálna integrácia [8] ukazujú, že overovanie správnosti kódu môže byť priebežnou súčasťou implementácie a pravidelnej integrácie zmien.

Testovanie má zároveň svoje prirodzené hranice. Dijkstra v tejto súvislosti uvádza, že testovanie programu môže ukázať prítomnosť chýb, ale nikdy nie ich neprítomnosť [7, s. 66]. Z tohto tvrdenia vyplýva, že ani dôkladné testovanie nezaručuje bezchybnosť systému. Aj po úspešnom vykonaní veľkého počtu testovacích prípadov môžu v systéme zostať neodhalené chyby. Testovanie preto neslúži ako dôkaz korektnosti, ale skôr ako prostriedok na zvýšenie miery dôvery v správanie systému.

Keďže v reálnych softvérových systémoch nie je možné pokryť celý priestor vstupov, stavov a chybových situácií, testovanie sa v praxi kombinuje s ďalšími technikami overovania kvality. Formálne metódy sa využívajú najmä pri verifikácii požiadaviek a návrhov kritických častí systému, napríklad pri bezpečnostných požiadavkách [4]. Revízia kódu zachytáva problémy logickej a architektonickej povahy [4] a statická analýza odhaľuje chyby bez nutnosti spustenia programu [24]. Takto vzniká vrstvený prístup, ktorý znižuje pravdepodobnosť, že kritická chyba ostane neodhalená [4].

1.2 Štatistika a náklady spojené s kvalitou softvéru

Testovanie má v praxi povest' činnosti, na ktorú nie je dostatok času ani finančných prostriedkov. Organizácie si síce uvedomujú význam testovania, no náklady na kvalitu a chyby v produkcii nemusia byť systematicky merané, čo sťažuje obhájenie investícií do testovania [19]. Podľa State of Testing Report 2024 polovica organizácií dokonca vôbec nemeria náklady spojené s chybami, ktoré sa dostanú do produkcie [19, s. 28]. Takáto absencia metrík môže sťažovať obhájenie investícií do kvality. Výsledkom môže byť stav, keď sa chyby dostávajú až do produkčného prostredia, kde je ich odhalenie a odstránenie spravidla náročnejšie než v skorších fázach vývoja [16].

Consortium for Information and Software Quality odhaduje, že nízka kvalita softvéru stála americkú ekonomiku v roku 2022 najmenej 2,41 bilióna dolárov, pričom akumulovaný technický dlh dosiahol približne 1,52 bilióna dolárov [16, s. 3]. Najväčší podiel na tomto raste majú náklady spojené s kybernetickou kriminalitou, ktoré v roku 2022 dosiahli 1,44 bilióna dolárov [16, s. 48]. Správa zároveň ukazuje, že náklady nekvalitného softvéru sa neprejavujú iba priamymi nákladmi na odstránenie problémov, ale aj stratami spôsobenými výpadkami systémov, stratou dát, reputačným poškodením a poklesom dôvery zákazníkov [16].

Podľa správy World Quality Report 2024-25, ktorú každoročne vydávajú spoločnosti Capgemini, Sogeti a OpenText na základe prieskumu medzi 1 775 manažérmi z 33 krajín [5, s. 4], dosiahla priemerná miera automatizácie testovania v organizáciách hodnotu 44 % [5, s. 6]. V oblasti testovacej automatizácie report uvádza, že 29 % organizácií už plne integrovalo generatívnu umelú inteligencia do svojich procesov a ďalších 42 % aktívne skúma jej potenciál, čo spolu predstavuje 71 % organizácií [5, s. 21]. Zároveň však 64 % spoločností uvádza, že ich pri rozširovaní automatizácie obmedzuje zastaraná architektúra, a 57 % priznáva absenciu ucelenej stratégie testovacej automatizácie [5, s. 22, s. 24]. Treba dodať, že samotná automatizácia nezaručuje kvalitu. Ak sú automatizované testy navrhnuté nesprávne alebo pokrývajú nevhodné scenáre, môžu vytvárať falošný pocit istoty o kvalite systému.

1.3 Význam testovacieho procesu

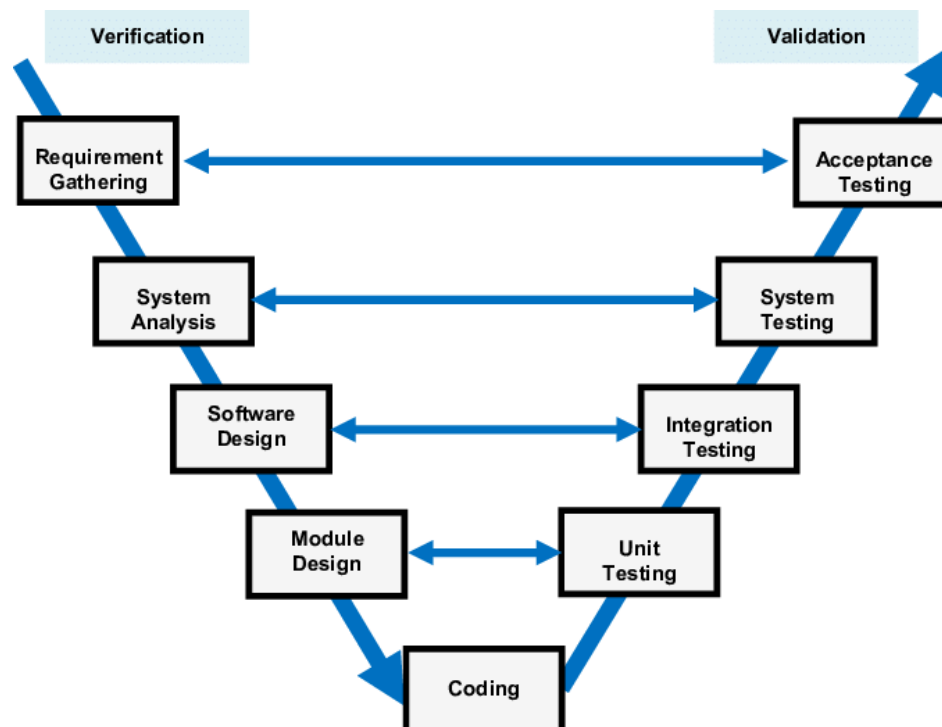
Cieľom testovacieho procesu je overiť správnosť a kvalitu softvéru. Testovací proces predstavuje súbor testovacích činností, ktoré sa podľa kontextu projektu môžu vykonávať postupne, iteratívne alebo paralelne [12]. Každá fáza má svoje opodstatnenie, od počiatočného plánovania cez návrh testov až po vykonávanie a analýzu ich výsledkov [12]. Takto štrukturovaný postup pomáha systematicky overovať jednotlivé časti systému a identifikovať chyby ešte pred jeho nasadením do produkcie. Správne nastavený testovací proces pomáha odhaľovať defekty pred uvedením softvéru do používania [24].

Včasná odhaľovanie problémov zároveň môže znižovať náklady na ich opravu, keďže problémy identifikované v skorších fázach vývoja bývajú spravidla lacnejšie na odstránenie než problémy nájdené neskôr [24].

Vzťah medzi vývojovými a testovacími aktivitami možno znázorniť pomocou modelov vývoja softvéru, v ktorých testovanie nadväzuje na jednotlivé fázy návrhu a implementácie systému.

Automatizované testovanie a používané nástroje

Medzi často používané nástroje automatizovaného testovania patrí **Selenium**, ktorý slúži na automatizáciu funkčných testov webových aplikácií [11]. **JUnit** je framework určený na jednotkové testovanie [10], zatiaľ čo **TestNG** podporuje v jazyku Java viaceré typy testovania a pokročilejšiu konfiguráciu testov [3]. **Cypress** sa využíva pri automatizovanom end-to-end testovaní frontendových webových aplikácií [17]. **Postman** sa používa pri vývoji, testovaní a ladení API rozhraní [26]. Jednotlivé nástroje možno zaradiť k rôznym úrovniam testovania, keďže SWEBOK rozlišuje testovanie samostatných modulov, skupín modulov a celého systému, teda najmä jednotkové, integračné a systémové testovanie [4].



Obr. 1.1: V-model v kontexte vývoja a testovania softvéru

Tabuľka 1.1: Zjednodušený prehľad fáz testovacieho procesu podľa ISTQB a SWEBOK [12, 4]

Fáza	Popis
1. Plánovanie testovania	Definovanie cieľov, rozsahu testovania, potrebných zdrojov a harmonogramu.
2. Návrh testovacích prípadov	Príprava testovacích scenárov, výber testovacích dát a stanovenie očakávaných výsledkov.
3. Vykonávanie testov	Realizácia testov, spúšťanie manuálnych alebo automatizovaných testov a zaznamenávanie výsledkov.
4. Vyhodnotenie výsledkov	Analýza úspešnosti testov, porovnanie skutočných a očakávaných výsledkov a identifikácia chýb.
5. Reportovanie a uzavretie	Vypracovanie testovacej správy, vyhodnotenie splnenia cieľov testovania a odporúčanie k pripravenosti produktu.

1.4 Charakteristiky kvality softvéru podľa ISO/IEC 25010

Pri testovaní softvéru nestačí overiť, či program „funguje“. Otázka znie skôr tak, či funguje správne, spoľahlivo, bezpečne a dostatočne rýchlo aj pod záťažou. Testovanie sa preto zameriava na kontrolu viacerých vlastností, ktoré určujú pripravenosť systému na reálne nasadenie. Medzi oblasti, ktoré je potrebné overiť, patria charakteristiky kvality softvérového produktu definované modelom ISO/IEC 25010 [13]:

- funkčná vhodnosť - či softvér vykonáva všetky požadované funkcie podľa špecifikácií
- spoľahlivosť — či systém funguje stabilne a bez chýb počas dlhodobého používania
- použiteľnosť — či sú ovládanie a používateľské rozhranie zrozumiteľné pre cieľového používateľa
- výkonnostná efektívnosť — či systém reaguje v primeranom čase a zvláda požadovanú záťaž
- kompatibilita — či softvér dokáže pracovať s rôznymi prostrediami, zariadeniami a verziami operačných systémov
- bezpečnosť — či sú dáta a prístupové oprávnenia dostatočne chránené pred zneužitím

- udržiavateľnosť — či je kód prehľadný, ľahko rozširiteľný a opraviteľný v budúcnosti
- prenositeľnosť — či je softvér možné nasadiť v rôznych prostrediach a platformách

Kapitola 2

Analýza existujúcich softvérových riešení

2.1 Čo je chatbot

Chatbot je program, ktorý simuluje konverzáciu s človekom [27]. V praxi funguje ako digitálny asistent, ktorý odpovedá na otázky, pomáha pri riešení problémov, dokáže používateľa naviesť správnym smerom bez priameho prezradenia odpovede, alebo slúži ako prvá línia zákazníckej podpory.

Súčasný chatboty nadväzujú na vývoj konverzačných programov, ktorý siaha do 60. rokov 20. storočia. Jedným z najstarších a najznámejších príkladov je program ELIZA, ktorý Joseph Weizenbaum predstavil v roku 1966 ako systém umožňujúci určitý typ komunikácie v prirodzenom jazyku medzi človekom a počítačom [25]. ELIZA simulovala psychoterapeutický rozhovor pomocou jednoduchých skriptov, ktoré v texte používateľa vyhľadávali kľúčové slová a podľa nich formulovala odpovede. O skutočné porozumenie prirodzenému jazyku však nešlo.

Výrazný posun vo vývoji chatbotov priniesol rozvoj spracovania prirodzeného jazyka a strojového učenia. Chatboty sa postupne zdokonalili v schopnosti viesť komplexnejší dialóg a pružnejšie reagovať na vstupy používateľa [27]. Súčasný systémy dokážu viesť plynulé rozhovory a prispôbujú odpovede na základe predchádzajúcich interakcií. Využívajú sa v zákazníckej podpore, bankovníctve, vzdelávaní aj v online službách [27, 20].

Jedným z dôvodov ich rozšírenia je ich nepretržitá dostupnosť. Chatbot môže komunikovať s používateľom kedykoľvek, bez ohľadu na pracovný čas a bez potreby okamžitej účasti ľudského operátora [20].

V zákazníckej podpore odpovedajú chatboty na často kladené otázky, asistujú pri výbere produktov a usmerňujú používateľov pri podávaní reklamácií. V bankovníctve poskytujú informácie, odpovedajú na otázky klientov a podporujú komunikáciu medzi

bankou a zákazníkom [20].

Špecifické uplatnenie majú chatboty aj v zdravotníctve. Môžu poskytovať základné informácie, pomáhať pri plánovaní návštev a ponúkať odporúčania zohľadňujúce zdravotnú históriu pacienta. Môžu tiež vykonávať základné predbežné posúdenie príznakov a nasmerovať pacienta na vhodný typ starostlivosti [27, 18].

Medzi hlavné výhody chatbotov v medicíne patrí:

Personalizované zdravotné poradenstvo: Chatboty môžu poskytovať zdravotné informácie, podporovať monitorovanie zdravotného stavu a asistovať pri orientačnom posúdení zdravotných potrieb pacienta [27].

Triáž a nepretržitá podpora: Chatboty sa môžu využívať pri podpore pacienta, monitorovaní zdravotného stavu a orientačnom posúdení príznakov [27]. Ich výhodou je aj možnosť poskytovať digitálnu podporu pri vyhľadávaní zdravotných informácií a orientačnom riešení menších zdravotných ťažkostí [18].

Plánovanie schôdzok: Zdravotnícke chatboty môžu podľa predchádzajúcich štúdií asistovať pri plánovaní lekárskeho návštev, vyhľadávaní zdravotníckych zariadení a poskytovaní informácií o liekoch [18].

2.2 ChatGPT ako príklad moderného chatbota

Medzi známe AI chatboty patrí **ChatGPT**. Je založený na veľkom jazykovom modeli, ktorý dokáže spracovať vstupy v prirodzenom jazyku a generovať textové odpovede [1]. Modely ako GPT-4 sú predtrénované na predikciu ďalšieho tokenu v texte s využitím verejne dostupných a licencovaných dát, ich správanie sa následne upravuje aj pomocou spätnej väzby od ľudských hodnotiteľov, napríklad metódou reinforcement learning from human feedback (RLHF) [1]. Vďaka tomu dokážu spracovať pokyny v prirodzenom jazyku a generovať súvislé textové odpovede v rôznych kontextoch. V praxi sa ChatGPT používa najmä na podporu komunikácie, tvorbu textového obsahu, vysvetľovanie pojmov, analýzu informácií a automatizáciu vybraných úloh.

V oblasti vzdelávania môže ChatGPT slúžiť ako pomôcka pri učení, vysvetľovaní pojmov a príprave študijných materiálov. Študentom umožňuje pracovať s informáciami interaktívnou formou a podporuje samostatné štúdium. Integrácia veľkých jazykových modelov do výučby zároveň otvára priestor na personalizáciu učebného procesu a poskytovanie spätnej väzby podľa individuálnych potrieb študentov [15]. Treba však zohľadniť aj riziká spojené s ich používaním, napríklad možnosť nepresných odpovedí, nekritické preberanie vygenerovaného obsahu alebo otázky akademickej integrity [15].

Okrem vzdelávania nachádza ChatGPT uplatnenie aj v prostredí firiem a organizácií, kde sa využíva najmä na podporu zákazníckej komunikácie, tvorbu dokumentácie, sumarizáciu textov alebo vyhľadávanie informácií v interných znalostných systémoch.

V zákazníckom servise môžu chatboty rýchlo reagovať na často kladené otázky, poskytovať základné informácie a znižovať záťaž pracovníkov podpory. Zvyčajne sa uplatňujú pri opakujúcich sa požiadavkách zákazníkov, napríklad pri poskytovaní informácií, odpovedaní na otázky, podpore nákupného procesu alebo komunikácii medzi organizáciou a zákazníkom [20].

ChatGPT môže nachádzať svoje uplatnenie aj v každodennom živote bežných používateľov. Môže slúžiť ako digitálny asistent pri získavaní rýchlych odpovedí na všeobecné otázky, vysvetľovaní neznámych pojmov, plánovaní denných aktivít alebo organizácii času. V bežnej praxi sa používa aj ako podpora pri písaní textov, napríklad e-mailov, motivačných listov alebo formálnych dokumentov. Používateľom môže pomôcť s formuláciou viet, kontrolou gramatiky a štylistiky, prekladom textov alebo učením sa cudzích jazykov.

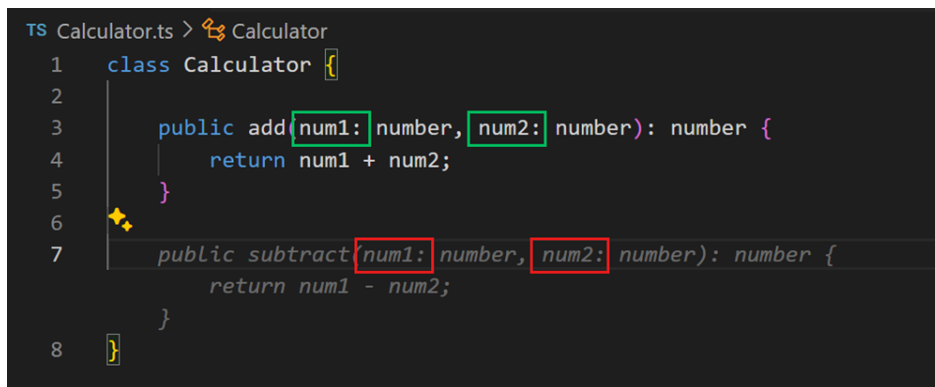
ChatGPT môže pomáhať aj pri riešení praktických problémov každodenného života. Používatelia ho môžu využiť napríklad pri hľadaní receptov, príprave cestovných plánov, získavaní základných informácií o destináciách alebo pri osvojovaní si nových zručností. Tieto možnosti vyplývajú najmä zo schopnosti modelu spracovať textové vstupy používateľa a generovať odpovede v prirodzenom jazyku podľa zadaného kontextu [1].

Na základe uvedených príkladov možno ChatGPT zaradiť medzi univerzálne použiteľné AI chatboty s využitím v rôznych oblastiach každodenného aj profesionálneho života. Jeho výhodou je najmä flexibilita a schopnosť reagovať na rôzne typy požiadaviek v prirodzenom jazyku. Zároveň však nenahrádza odborné služby a jeho odpovede je potrebné kriticky posudzovať, najmä v oblastiach, kde môže mať nesprávna informácia vážne dôsledky [15, 1]. V súčasnosti existuje viacero ďalších AI chatbotov a inteligentných nástrojov navrhnutých pre konkrétne oblasti, ako je technická podpora, zákaznícky servis, programovanie, zdravotníctvo alebo vzdelávanie. Špecializované riešenia môžu byť v úzko vymedzených úlohách presnejšie alebo efektívnejšie. ChatGPT však možno chápať ako všeobecne použiteľný nástroj založený na veľkom jazykovom modeli, pričom modely ako GPT-4 dokážu spracovať textové vstupy a generovať textové odpovede v širokom spektre kontextov a aplikácií [1].

2.3 GitHub Copilot

GitHub Copilot je AI nástroj určený na podporu vývoja softvéru, ktorý funguje ako inteligentný asistent priamo v integrovanom vývojovom prostredí. V odbornej literatúre sa označuje aj ako nástroj typu *AI pair programmer*, pretože dokáže na základe kontextu, napríklad komentárov, názvov metód alebo okolitého kódu, generovať návrhy zdrojového kódu v rôznych programovacích jazykoch [6]. Nástroj je založený na pro-

dukčnej verzii modelu Codex a jeho účelom je podporiť vývojárov najmä pri generovaní návrhov zdrojového kódu na základe kontextu aktuálnej úlohy [6].



```
TS Calculator.ts > Calculator
1 class Calculator {
2
3   public add(num1: number, num2: number): number {
4     return num1 + num2;
5   }
6
7   public subtract(num1: number, num2: number): number {
8     return num1 - num2;
9   }
10 }
```

Obr. 2.1: Návrh kódu od GitHub Copilot [9]

V praxi sa Copilot využíva na podporu produktivity vývojárov. Okrem automatického dopĺňania riadkov kódu dokáže na základe stručného popisu navrhovať funkcie, triedy, testy alebo komentáre. Empirické hodnotenia však ukazujú, že hoci nástroj generuje použiteľné riešenia, niektoré jeho návrhy môžu byť chybné alebo nereprodukovateľné, a preto si vyžadujú kontrolu a kritické posúdenie vývojárom [6].

Copilot môže urýchliť návrh základných testovacích prípadov, keďže dokáže generovať návrhy testovacieho kódu na základe existujúceho kódu alebo textového zadania [9]. Manuálna tvorba testov si vyžaduje znalosť testovanej funkcionality, očakávaných výsledkov a možných hraničných stavov. Copilot dokáže časť tejto prípravy automatizovať, čím vývojárovi uvoľňuje priestor na riešenie zložitejších úloh, akými sú návrh testovacej stratégie, kontrola pokrytia alebo vyhodnotenie správnosti vygenerovaných testov.

V softvérovom testovaní zohráva Copilot podpornú úlohu pri tvorbe a rozširovaní testovacieho kódu. Na základe existujúcich tried, funkcií alebo komentárov dokáže navrhnúť jednotkové a integračné testy [9]. Pri vhodne formulovanom zadaní vie navrhnúť testy pre rôzne scenáre vrátane okrajových prípadov, spracovania výnimiek a validácie vstupných dát [9]. Takýto prístup môže podporiť pravidelnejšie písanie testov a znížiť čas potrebný na prípravu opakujúcich sa častí testovacieho kódu.

Tento nástroj sa dá využiť aj pri údržbe existujúcich testov, napríklad pri dopĺňaní nových testovacích prípadov po úpravách zdrojového kódu alebo pri návrhu zmien v existujúcich testoch [9]. V kontexte regresného testovania môže pomôcť pri rýchlejšej príprave testov, ktorých cieľom je overiť, či nové zmeny nenarušili existujúcu funkcionality aplikácie.

Napriek uvedeným výhodám GitHub Copilot nedokáže plne nahradiť komplexné testovacie stratégie ani skúsenosti testerov a vývojárov. Testy, ktoré navrhuje, nemusia pokrývať všetky scenáre a preto je potrebné ich preveriť, spustiť, upraviť a doplniť

podľa požiadaviek konkrétneho projektu [9]. Efektívne využívanie Copilotu pri generovaní testov preto spočíva v prepojení automatizácie s odborným ľudským dohľadom.

2.4 Slackbot AI

Slack je cloudová platforma určená na podporu tímovej komunikácie, spolupráce a riadenia informácií v organizáciách [14]. Jej súčasťou je aj Slackbot, ktorý je v súčasnosti prezentovaný ako osobný AI agent zabudovaný priamo do prostredia Slacku. Slackbot pracuje s kontextom správ, kanálov, súborov a pripojených aplikácií, ku ktorým má používateľ prístup, a dokáže pomáhať pri vyhľadávaní informácií, sumarizácii obsahu, príprave podkladov alebo tvorbe textov [23, 22].

Podľa prieskumu medzi administrátormi Slacku uviedlo 32,0 % respondentov zvýšenie produktivity, pričom interná e-mailová komunikácia sa znížila o 48,6 % a počet stretnutí o 25,1 %. Zlepšenie tímovej kultúry uviedlo 79 % respondentov a viac než 80 % zaznamenalo vyššiu transparentnosť a jednoduchší prístup k informáciám [14].

Slackbot využíva najmä ako nástroj na podporu spolupráce v tímoch. Používateľom sprístupňuje informácie, pomáha pri orientácii v pracovných kanáloch a dokáže automatizovať jednoduché rutinné úlohy. Na rozdiel od univerzálnych chatbotov funguje priamo v prostredí pracovných kanálov, kde reaguje na otázky používateľa v kontexte aktuálnych diskusií, súborov a projektov [23, 22].

V prostredí softvérového vývoja a testovania zohráva Slackbot podpornú úlohu predovšetkým v komunikácii a koordinácii tímov. Keďže pôsobí priamo v pracovnom prostredí Slacku a pracuje s informáciami zo správ, súborov a pripojených aplikácií, prispieva k rýchlejšiemu vyhľadávaniu informácií, sumarizácii diskusií a orientácii v aktuálnom stave práce [23, 22]. Vývojári, tester a projektoví manažéri tak môžu jednoduchšie sledovať priebeh prác, zdieľať relevantné informácie a reagovať na vzniknuté problémy.

Pri testovaní softvéru môže byť Slackbot užitočný najmä ako komunikačný a informačný doplnok. V tímoch, ktoré využívajú Slack spolu s ďalšími pracovnými nástrojmi, môže Slack slúžiť ako miesto, kde sa sústreďujú informácie o stave úloh, zmenách v projekte alebo súvisiacich pracovných podkladoch. Takéto zdieľanie informácií môže podporiť rýchlejšiu reakciu členov tímu a znížiť riziko, že dôležité informácie ostanú nepovšimnuté [23, 22].

Slackbot zároveň môže prispieť k lepšej prehľadnosti testovacieho procesu tým, že umožňuje pracovať s informáciami priamo v komunikačných kanáloch tímu. Vývojári, tester a projektoví manažéri tak môžu jednoduchšie sledovať aktuálny stav projektu, diskutovať o chybách a koordinovať ďalšie kroky. Týmto spôsobom sa znižuje riziko nedorozumení a podporuje sa efektívnejšia spolupráca medzi členmi tímu počas vývoja

a testovania.

Na rozdiel od ChatGPT alebo GitHub Copilot, Slackbot neslúži primárne na generovanie zdrojového kódu ani testovacích prípadov. Jeho prínos spočíva najmä v podpore tímovej komunikácie, automatizácii jednoduchých pracovných procesov a v zdieľaní informácií. Práve tieto aspekty sú významné pri riadení vývojových projektov, pretože kvalita komunikácie v tíme ovplyvňuje koordináciu práce, rýchlosť riešenia problémov a plynulosť vývoja.

Slackbot teda predstavuje doplnkový nástroj k ostatným AI nástrojom používaným pri vývoji softvéru. Hoci jeho využitie nespočíva priamo v technickej tvorbe kódu alebo testov, jeho prínos v oblasti zdieľania informácií a koordinácie tímu môže nepriamo podporiť kvalitu a plynulosť vývoja aj testovania softvéru.

2.5 Porovnanie vybraných AI nástrojov

Vybrané nástroje sa líšia najmä účelom použitia a miestom, kde podporujú proces vývoja softvéru. ChatGPT predstavuje univerzálny konverzačný nástroj, GitHub Copilot sa zameriava na podporu písania kódu a testov priamo vo vývojovom prostredí, a Slackbot AI podporuje tímovú komunikáciu a prácu s informáciami v organizácii.

Tabuľka 2.1: Porovnanie vybraných AI nástrojov

Nástroj	Hlavné využitie	Prínos pri vývoji softvéru	Obmedzenie
ChatGPT	Konverzácia v prirodzenom jazyku, vysvetľovanie, tvorba textov	Podpora analýzy, dokumentácie, návrhu testovacích scenárov a vysvetľovania pojmov	Výstupy je potrebné kriticky overovať
GitHub Copilot	Návrhy kódu a testov vo vývojovom prostredí	Pomoc pri písaní funkcií, tried, jednotkových testov a opakujúcich sa častí kódu	Návrhy nemusia byť vždy správne alebo úplné
Slackbot AI	Práca s informáciami a komunikácia v prostredí Slacku	Podpora koordinácie tímu, sumarizácie diskusií a vyhľadávania informácií	Neslúži primárne na tvorbu kódu ani testovacích prípadov

Kapitola 3

Analýza problému a špecifikácia riešenia

3.1 Identifikácia problému

V oblasti softvérového testovania sa v praxi môže vyskytovať problém nedostatočne systematického prístupu, najmä v počiatočných fázach vývoja. Testovanie predstavuje jednu z kľúčových činností pri vývoji stabilného a kvalitného softvérového produktu [24]. Návrh testovacieho plánu a identifikácia relevantných oblastí testovania si vyžadujú nielen praktické skúsenosti a odborné znalosti, ale aj časové zdroje. Ak sa niektorý z uvedených aspektov opomenie, môže dôjsť k prehliadnutiu dôležitých oblastí testovania, ako sú nefunkčné požiadavky, bezpečnostné riziká či rizikovo významné scenáre, čo môže mať negatívny vplyv na kvalitu výsledného produktu [4, 12].

Tento problém sa výraznejšie prejavuje v menších vývojových tímoch a v prostrediach, kde testovanie vykonávajú menej skúsení tester. Tí nemusia disponovať uceleným pohľadom na testovanie, čo môže ovplyvniť technickú, časovú aj finančnú stránku projektu. Dôsledkom býva opakovanie chýb, presun testovacích aktivít do neskorších fáz vývoja alebo neúplné testovanie. Takýto stav zvyšuje riziko, že niektoré chyby останú neodhalené až do neskorších fáz vývoja alebo prevádzky, rozhodovanie o rozsahu testovania pritom súvisí aj s nákladmi a rizikami zostávajúcich zlyhaní [4, 24].

3.2 Cieľ navrhovaného riešenia

Na základe identifikovaného problému bolo navrhnuté riešenie vo forme chatbota, ktorý slúži ako podporný nástroj pri plánovaní testovacích procesov. Hlavným cieľom riešenia je automatizované generovanie testovacieho checklistu na základe informácií o konkrétnom softvérovom projekte.

Princíp fungovania chatbota je založený na textovej konverzácii s používateľom v

prírodzenom jazyku. Aby bol výstupný checklist použiteľný ako podklad na testovanie, chatbot získava základné informácie o projekte, napríklad typ aplikácie, použité technológie, cieľovú platformu, hlavné funkcie alebo špecifické požiadavky. Na základe týchto vstupov následne identifikuje relevantné oblasti, ktoré by mali byť pri testovaní zohľadnené.

Výstupom riešenia je štruktúrovaný checklist, ktorý môže používateľ ďalej upravovať a prispôbovať konkrétnym potrebám projektu.

3.3 Cieľová skupina používateľov

Navrhované riešenie je určené najmä pre používateľov, ktorí potrebujú pripraviť základný testovací checklist, no nemajú rozsiahle skúsenosti s návrhom testovacieho plánu. Typickú cieľovú skupinu predstavujú menšie vývojové tímy, tester, vývojári vykonávajúci základné overovanie vlastného kódu a študenti, ktorí sa oboznamujú so softvérovým testovaním.

Pre menej skúsených používateľov plní chatbot úlohu podporného nástroja pri orientácii v testovacom procese. Pomáha identifikovať oblasti, ktorým je potrebné venovať pri testovaní pozornosť, a zároveň mu umožňuje pracovať s návrhom checklistu bez potreby detailnej znalosti testovacích metodík alebo špecializovaných nástrojov.

3.4 Prínosy navrhovaného riešenia

Hlavným prínosom navrhovaného riešenia je jeho dostupnosť a nízka vstupná bariéra pre používateľa. Chatbot umožňuje rýchlo získať prehľad o oblastiach, ktoré je potrebné pri testovaní softvéru zohľadniť, bez nutnosti detailnej znalosti testovacích metodík. Vďaka konverzácii v prírodzenom jazyku je jeho použitie intuitívne a vhodné aj pre menej skúsených testerov či menšie vývojové tímy.

Konverzačné rozhranie zároveň umožňuje postupné spresňovanie požiadaviek. Používateľ môže priebežne dopĺňať informácie o projekte a chatbot na tieto vstupy reaguje úpravou alebo rozšírením generovaného checklistu. Takýto prístup podporuje iteratívne spresňovanie testovacích oblastí a umožňuje prispôbiť výstup konkrétnemu projektu.

Ďalším prínosom je flexibilita riešenia. Chatbot môže byť využitý pri rôznych typoch projektov, napríklad pri webových aplikáciách, mobilných aplikáciách alebo aplikačných rozhraniach (API). Systém prispôbuje výstup charakteristikám daného projektu. Pri testovaní mobilnej aplikácie zohľadňuje aspekty ako rôzne veľkosti obrazoviek, orientáciu zariadenia alebo spotrebu batérie, zatiaľ čo pri webových aplikáciách sa zameriava na kompatibilitu s rôznymi prehliadačmi a podporu responzívneho dizajnu.

Používanie chatbota v rámci testovacieho procesu môže slúžiť ako podporný nástroj

pri systematickejšom uvažovaní o testovaní už v počiatočných fázach projektu. Keďže návrh testovania zahŕňa identifikáciu testovacích oblastí, rizík, testovacích podmienok a vhodných scenárov, takýto nástroj môže pomôcť pripomenúť oblasti, ktoré by pri menej systematickom prístupe mohli zostať nepokryté [4, 12]. Tým môže nepriamo prispieť k zníženiu rizika, že sa niektoré chyby prejavajú až v neskorších fázach vývoja alebo po uvedení systému do produkcie. Problémy identifikované v skorších fázach vývoja bývajú spravidla lacnejšie na odstránenie než problémy nájdené neskôr [24].

Medzi prínosy patrí aj časová efektivita. Manuálna tvorba testovacieho plánu môže byť časovo náročná, keďže testovacie aktivity vyžadujú plánovanie zdrojov, dokumentácie, testovacieho prostredia, rizík a vhodných testovacích prípadov [4, 12]. Navrhované riešenie umožňuje vygenerovanie základného checklistu, ktorý môže slúžiť ako východiskový podklad pre ďalšie plánovanie testovania a ktorý možno následne upravovať a rozširovať podľa špecifických potrieb konkrétneho projektu.

Z praktického hľadiska riešenie umožňuje vývojovým tímom jednoduchšie aplikovať základné testovacie postupy bez nutnosti absolvovať časovo náročné školenia, kurzy alebo štúdium odbornej literatúry. Chatbot v tomto kontexte pôsobí ako podporný nástroj poskytujúci základné usmernenie a štruktúru pre testovací proces.

Systém môže byť využitý aj ako edukačný nástroj, najmä pre používateľov s menšími skúsenosťami v oblasti testovania softvéru. Pri generovaní checklistu môže chatbot vysvetľovať jednotlivé oblasti, ich význam a spôsob, akým ich možno overiť. Používateľ tak postupne získava prehľad o základných postupoch v testovaní a rozširuje si praktické znalosti.

3.5 Funkčné a nefunkčné požiadavky

3.5.1 Funkčné požiadavky

Funkčné požiadavky predstavujú operácie a služby, ktoré má systém vykonávať, aby zabezpečil podporu testovania softvéru. Vychádzajú z identifikovaných potrieb používateľa a definujú očakávanú funkcionálnu kapacitu systému.

Vedenie konverzácie s používateľom

- Chatbot musí byť schopný viesť prirodzenú konverzáciu v angličtine
- Systém musí klásť relevantné otázky zamerané na získanie potrebných informácií o projekte
- Chatbot musí dokázať spracovať voľný textový opis projektu
- V prípade nejasností alebo neúplností si chatbot vyžiada doplnenie informácií

Udržiavanie kontextu konverzácie

- Systém musí uchovávať kontext celej konverzácie, vrátane predchádzajúcich odpovedí používateľa
- Chatbot musí nadväzovať na predchádzajúce informácie a neklásť opakované otázky
- Systém musí rozpoznať vzťahy medzi jednotlivými časťami projektu a využívať tieto informácie pri generovaní checklistu.

Spracovanie textového opisu projektu

- Chatbot musí analyzovať vstupný opis projektu a extrahovať z neho kľúčové informácie, ako sú typ aplikácie, hlavné funkcionality, použité technológie a cieľová skupina používateľov
- Okrem explicitne uvedených informácií musí chatbot identifikovať aj informácie odvodené z kontextu, napríklad určiť na základe typu aplikácie relevantné oblasti potrebné na testovanie
- Chatbot musí rozpoznať technickú terminológiu, napríklad REST API, responzívny dizajn či autentifikácia

Identifikácia oblastí na testovanie

- Systém musí na základe opisu projektu od používateľa identifikovať relevantné testovacie oblasti vrátane funkčného testovania
- Chatbot musí rozlišovať medzi všeobecnými testovacími oblasťami aplikovateľnými na väčšinu softvérových projektov a špecifickými oblasťami, ktoré vyplývajú z konkrétnych charakteristík projektu
- Systém zároveň musí určiť priority jednotlivých testovacích oblastí s ohľadom na typ projektu

Tvorba štrukturovaného checklistu

- Chatbot musí vygenerovať prehľadný a logicky štrukturovaný checklist obsahujúci identifikované testovacie oblasti
- Každá položka checklistu musí obsahovať stručný opis príslušnej testovacej oblasti spolu s vysvetlením jej významu v kontexte hodnoteného projektu.
- Checklist musí obsahovať testovanie usporiadané podľa ich relevantnosti a priority pre konkrétny projekt

3.5.2 Nefunkčné požiadavky

Okrem funkčných požiadaviek boli definované aj nefunkčné požiadavky zamerané na kvalitu a použiteľnosť riešenia:

Intuitívnosť

- Komunikačné rozhranie musí byť zrozumiteľné a použiteľné bez nutnosti zaškolenia
- Systém nesmie vyžadovať predchádzajúce znalosti testovania ani odbornej terminológie
- Chatbot musí poskytovať jasné a zrozumiteľné inštrukcie

Dostupnosť

- Systém musí byť voľne dostupný používateľom s rôznou úrovňou skúseností v oblasti testovania softvéru
- Chatbot musí byť prístupný cez bežné webové rozhranie

Efektivita

- Systém musí minimalizovať počet otázok potrebných na získanie relevantných informácií o projekte
- Navrhované riešenie má optimalizovať využitie systémových prostriedkov tak, aby nedochádzalo k nadmernému zaťaženiu serverového prostredia

Kapitola 4

Návrh systému

4.1 Prehľad navrhovaného riešenia

Navrhovaný systém je konverzačný nástroj, ktorého úlohou je pomôcť používateľovi pri zostavení testovacieho checklistu pre konkrétny softvérový projekt. Systém postupne získava informácie o projekte prostredníctvom otázok formulovaných v prirodzenom jazyku, pričom zohľadňuje typ aplikácie, použité technológie, cieľovú platformu, spôsob autentifikácie, prácu s dátami, integrácie s externými službami a ďalšie relevantné vlastnosti.

Získané odpovede sa priebežne ukladajú do internej štruktúry projektu. Na základe týchto údajov systém identifikuje relevantné testovacie oblasti a vygeneruje štruktúrovaný checklist. Výstupom nie je úplný testovací plán, ale podklad, ktorý môže používateľ ďalej upraviť, rozšíriť alebo použiť ako východisko pri plánovaní testovacích aktivít.

4.2 Otázky a odpovede chatbota

Aby chatbot dokázal viesť s používateľom zmysluplnú a plynulú konverzáciu, je potrebné vopred definovať okruh otázok, ktoré bude klásť. Tieto otázky slúžia na získanie kľúčových informácií o projekte a zároveň pomáhajú udržiavať kontext celej komunikácie. Konverzácia s chatbotom prebieha v dvoch fázach. V prvej fáze systém identifikuje typ aplikácie, napríklad webovú, mobilnú, desktopovú aplikáciu alebo API. Na základe tejto informácie následne zvolí príslušnú sadu otázok. V druhej fáze systematicky prechádza jednotlivé oblasti a zbiera informácie potrebné pre generovanie checklistu.

4.2.1 Určenie typu aplikácie

Prvou otázkou je vždy typ aplikácie:

- „O aký typ aplikácie ide? (webová aplikácia, mobilná aplikácia, desktopová aplikácia alebo API / backendová služba)“

Odpoveď na túto otázku určuje, akú sadu otázok chatbot používateľovi následne položí. Každý typ aplikácie má vlastnú schému otázok, pretože relevantné testovacie oblasti sa medzi typmi líšia. Pri webovej aplikácii sa otázky zameriavajú na responzivnosť a podporu prehliadačov, pri mobilnej aplikácii na cieľovú platformu a oprávnenia zariadenia, pri desktopovej aplikácii na inštaláciu a integráciu s operačným systémom a pri API na endpointy, štruktúru odpovedí a bezpečnosť. Oblasť, ktorá je pre jeden typ aplikácie zásadná, nemusí byť pre iný typ aplikácie relevantná.

4.2.2 Všeobecný kontext projektu

Niekoľko otázok je spoločných pre všetky typy aplikácii: názov aplikácie, jej účel a cieľová skupina používateľov. Ďalšie otázky sa už diferencujú podľa typu aplikácie.

Pri webovej aplikácii chatbot zisťuje, či je verejne dostupná, a žiada URL adresy prostredí - vývojového, testovacieho a produkčného. Pri mobilnej aplikácii sú podstatné cieľové platformy (Android, iOS alebo obe) a minimálne verzie operačných systémov, na ktorých má aplikácia fungovať. Pri desktopovej aplikácii je dôležité zistiť cieľové operačné systémy a podporované architektúry procesora. Aplikácia môže fungovať odlišne na architektúrach x86 a ARM, čo sa pri plánovaní testov ľahko prehliadne. Pri API chatbot zisťuje jeho typ (REST, GraphQL, gRPC), keďže každý z týchto typov má odlišné vzory testovania. Napríklad pri GraphQL je potrebné testovať hĺbku a komplexitu dotazov (*query depth* a *query complexity*), aby sa predišlo problémom s výkonom servera.

4.2.3 Podmienené otázky

Väčšina otázok je podmienená, chatbot sa pýta len na oblasti, ktoré sa danej aplikácie naozaj týkajú. Postup je nasledujúci: chatbot najprv zistí, či aplikácia má určitú vlastnosť. Ak ju nemá, celý blok otázok týkajúcich sa danej vlastnosti sa preskočí. Ak áno, nasledujú otázky na upresnenie detailov. Každý typ aplikácie má vlastnú množinu podmienených blokov.

Autentifikácia a správa prístupov

Chatbot sa najprv opýta, či aplikácia vyžaduje prihlásenie. Ak nie, autentifikácia sa ďalej nerieši.

Ak áno, nasledujú otázky o spôsobe prihlásenia, napríklad email a heslo, OAuth, SSO. Mobilná aplikácia má v tomto bloku navyše otázku týkajúcu sa biometrickej autentifikácie. Webová aplikácia rieši správu súbežných používateľských prihlásení a admi-

nistrátorské oprávnenia. API sa namiesto toho zameriava na rozsahy oprávnení, dobu platnosti tokenov a možnosť ich odvolania.

Dáta a formuláre

Chatbot zisťuje, ako aplikácia pracuje s dátami: formuláre, nahrávanie a sťahovanie súborov. Mobilná aplikácia má v tomto bloku tiež navyše otázku o offline režime. Ak aplikácia funguje bez pripojenia internetu, je potrebné otestovať jej správanie pri výpadku pripojenia, synchronizáciu po jeho obnovení a konzistenciu lokálneho úložiska. Desktopová aplikácia sa zameriava na prácu so súbormi podrobnejšie: podporované formáty, možnosti importu a exportu dát, spracovanie veľkých súborov, ako aj implementáciu mechanizmov *undo/redo* pre vrátenie alebo opakovanie vykonaných operácií.

Nefunkčné požiadavky a prostredie

Systém zisťuje výkonnostné požiadavky a očakávanú záťaž. Pri webovej aplikácii ide o počet súbežných používateľov, pri API o počet požiadaviek za sekundu. Okrem toho chatbot zisťuje používanie cache mechanizmov, požiadavky na SEO a potreby v oblasti analytiky.

Pri mobilnej aplikácii je dôležité zohľadniť testovanie na zariadeniach s minimálnou konfiguráciou. Aplikácia, ktorá dobre funguje na výkonnom vývojárskom zariadení, nemusí korektne fungovať na starších modeloch mobilných zariadení.

4.2.4 Integrácie a externé služby

Väčšina aplikácií sa pri niektorých funkciách spolieha na externé služby. Platobné brány, odosielanie e-mailov, prihlasovanie cez Google alebo webhooky pre notifikácie patria medzi miesta, kde môže pri integrácii dôjsť k chýbám a kde testovanie bez znalosti konkrétnej integrácie nestačí.

Chatbot sa preto pýta, či aplikácia využíva platobné brány (napríklad Stripe alebo PayPal), e-mailové alebo SMS notifikácie, externých autentifikačných poskytovateľov alebo integrácie so službami tretích strán. Pri každej potvrdennej integrácii sa otázky upresňujú. Pri platobnej bráne je napríklad podstatné zistiť, či aplikácia spracúva čísla kariet priamo, alebo celý proces prebieha na strane poskytovateľa.

Tieto informácie majú priamy dopad na výsledný checklist. Ak aplikácia odosiela e-maily, je potrebné otestovať jej správanie pri nedostupnosti servera SMTP. Ak využíva webhooky, je potrebné overiť, čo sa stane pri opakovanom doručení tej istej udalosti. Bez týchto otázok by checklist oblasť integrácii vynechal.

4.2.5 otázky špecifické pre API

Pre API aplikácie je sada otázok výrazne odlišná od ostatných typov. Keďže API neposkytuje vizuálne rozhranie, testovacie oblasti sa posúvajú smerom k správaniu endpointov, formátu odpovedí a bezpečnosti komunikácie.

Chatbot zisťuje typ API (REST, GraphQL alebo gRPC), pretože každý z nich funguje odlišne. Pri GraphQL je potrebné testovať hĺbku a komplexitu dotazov, aby sa predišlo preťaženiu servera. Pri REST API sa chatbot pýta na štruktúru endpointov, používané HTTP metódy a správanie pri rôznych stavových kódach.

Autentifikácia API má vlastný blok otázok: aký spôsob autentifikácie sa používa (API kľúč, OAuth2, JWT alebo Basic Auth), ako dlho platia tokeny a či je možné ich odvolať pred vypršaním. Chatbot ďalej zisťuje, či API využíva mechanizmy *rate limiting* a ako sa správa pri ich prekročení.

Pri webhookoch sa otázky zameriavajú na overenie identity odosielateľa, spracovanie duplicitných udalostí a správanie pri nedostupnosti prijímajúceho servera. Verziovanie predstavuje samostatnú oblasť. Ak API podporuje viacero verzií súčasne, je potrebné overiť, že staršie verzie fungujú podľa dokumentácie a že zmeny v novej verzii nenarušili funkčnosť existujúcich klientov.

4.3 Spracovanie odpovedí a udržiavanie kontextu

Chatbot uchováva celý priebeh konverzácie. Neopakuje otázky, nadväzuje na predchádzajúce odpovede a ak je niečo nejasné, vyžiada si upresnenie skôr, než pokračuje ďalej. Ak používateľ neskôr zmení alebo upresní niektorú z predchádzajúcich odpovedí, chatbot túto zmenu zaznamená a zohľadní pri ďalšom postupe. Zozbierané informácie ukladá do internej štruktúry zodpovedajúcej typu aplikácie a po skončení konverzácie ich použije na vygenerovanie checklistu. Používateľ o tejto štruktúre nevie. Z jeho pohľadu ide o bežný rozhovor. Oblasti, ktoré označil ako nerelevantné, sa vo výslednom checkliste neobjavia.

4.4 Generovanie checklistu

Po získaní odpovedí na všetky relevantné otázky chatbot pristúpi ku generovaniu checklistu. Zozbierané informácie sú v tomto bode uložené v internej schéme projektu, kde každé pole obsahuje konkrétnu hodnotu zadanú používateľom.

Systém vyhodnotí sadu pravidiel, ktoré porovnávajú hodnoty v schéme s definovanými rizikovými podmienkami. Ak napríklad aplikácia využíva autentifikáciu a definuje viacero používateľských rolí, aktivuje sa pravidlo pre testovanie autorizácie — overenie, či používateľ s obmedzenými oprávneniami nedokáže pristupovať k funkciám vyhrade-

ným pre administrátorov. Každému pravidlu je priradená priorita. Vysoká, stredná alebo nízka.

Výsledné položky sú zoradené podľa priority a zoskupené do oblastí. Checklist obsahuje konkrétne testovacie scenáre, nie všeobecné odporúčania. Pri webovej aplikácii s nahrávaním súborov tak checklist obsahuje napríklad testovanie prípadov, kde používateľ nahrať súbor s nebezpečnou príponou alebo súbor výrazne presahujúci povolený limit veľkosti.

Kapitola 5

Implementácia riešenia

Systém sa skladá z troch častí: frontend, backend a databáza. **Frontend** je jednostránková React aplikácia, ktorá tvorí chatové rozhranie. Používateľ v nej okrem okna konverzácie vidí aj bočný panel so zoznamom predošlých konverzácií, dokončené konverzácie sú označené zelenou farbou. Súčasťou frontendu je aj prihlasovací a registračný formulár.

Serverová časť systému **backend** je napísaná v Pythone s využitím frameworku FastAPI. Backend prijíma požiadavky z frontendu, riadi priebeh konverzácie a komunikuje s jazykovými modelmi.

Na ukladanie dát slúži **Supabase**, čo je spravovaná PostgreSQL databáza, v ktorej sa uchovávaajú všetky relácie (*sessions*) a správy. Supabase zároveň poskytuje vstavanú autentifikačnú službu, ktorá zabezpečuje správu používateľských účtov a vydávanie prihlasovacích tokenov.

Výmena dát medzi frontendom a backendom prebieha cez REST rozhranie nad protokolom HTTP. Stav konverzácie uchováva výhradne server, frontend si žiadne dáta lokálne neukladá a všetko potrebné si vyžiada od servera.

5.1 Architektúra

API vrstva

API tvoria dva moduly: `routes/chat.py` a `routes/sessions.py`.

Endpoint `POST /chat` prevezme správu od používateľa, zapíše ju do databázy, zavolá Orchestratora a klientovi vráti vygenerovanú odpoveď spolu s príznakom dokončenia relácie. Modul `sessions.py` obsahuje endpointy na vytvorenie novej relácie, získanie zoznamu relácií používateľa, načítanie histórie konkrétnej relácie, jej odstránenie a stiahnutie dokončeného checklistu vo formáte XLSX. Každá z týchto ciest vyžaduje závislosť `get_current_user_id`, ktorá overí JWT token a získa identifikátor prihláseného používateľa. API vrstva samo o sebe neimplementuje doménovú logiku. Jej zodpoved-

nosťou je preklad HTTP požiadaviek na volania nižších služieb a prevod doménových výnimiek na zodpovedajúce HTTP stavové kódy.

Autentifikácia

Autentifikácia je implementovaná v module `core/auth.py` s využitím služby Supabase Auth. Hlavnou súčasťou je funkcia `get_current_user_id`, ktorá je vo FastAPI použitá ako závislosť pri všetkých endpointoch. Z hlavičky `Authorization: Bearer ...` načíta JWT token, overí ho voči službe Supabase a vráti identifikátor prihláseného používateľa. Ak je token neplatný alebo má vypršanú platnosť, funkcia vráti odpoveď s HTTP kódom 401. Takéto riešenie oddeľuje overovanie identity od ostatnej aplikačnej logiky. Backend sám neuchováva heslá, túto zodpovednosť plne preberá Supabase. Získaný identifikátor používateľa sa následne používa pri ukladaní a načítavaní relácií z databázy, čím sa zabezpečí, že každý používateľ má prístup výhradne k vlastným rozhovorom.

Orchestrator

Orchestrator (`core/orchestrator.py`) zabezpečuje spracovanie jednej správy v rámci konverzácie. Po prijatí správy najprv načíta stav relácie z databázy. Ak typ aplikácie ešte nie je známy, klasifikáciu vykoná RouterService a z AgentRegistry sa vyberie zodpovedajúci špecialista. ValidationService potom z doterajšej konverzácie extrahuje známe fakty, porovná ich so schémou a zistí, ktoré polia ešte chýbajú. Na základe tohto zistenia sa zavolá buď ConversationalAgent, ktorý sformuluje ďalšiu otázku, alebo ChecklistGenerator, ktorý vytvorí výsledný checklist. Aktualizovaný stav sa nakoniec uloží späť do databázy.

Trieda Orchestrator sa nevytvára priamo v kóde, ale prostredníctvom továrenskej funkcie `build_orchestrator`. Táto funkcia poskladá objektový graf, priradí jednotlivým agentom konkrétne jazykové modely a vloží závislosti do objektu `OrchestratorServices`. Takáto štruktúra umožňuje vymeniť ktorýkoľvek model bez zásahu do logiky agentov.

AgentRegistry

AgentRegistry (`core/agent_registry.py`) funguje ako slovník, ktorý k jednotlivým hodnotám `AppType` priradzuje konkrétneho špecialistu. Pri inicializácii sa vytvoria inštancie všetkých piatich špecialistov (`WebAgent`, `MobileAgent`, `DesktopAgent`, `APIAgent`, `OtherAgent`), ktoré sa následne použijú počas behu aplikácie. Mapovanie by bolo možné implementovať aj pomocou bloku `if-else`, vyčlenenie do samostatnej triedy však uľahčuje pridávanie ďalších špecialistov, keďže ich stačí zaregistrovať na jednom mieste.

SessionState a SessionManager

`SessionState` je dataclass definovaná v module `core/session_state.py` a slúži na uchovanie stavu konverzácie počas obsluhy jednej HTTP požiadavky. Skladá sa z typu aplikácie, aktuálnej schémy, histórie správ, mena aktívneho agenta a príznaku do-

končenia. Keďže ide o objekt uchovávaný v pamäti, jeho databázovým modelom je `SessionModel` a prevod medzi nimi zabezpečuje funkcia `db_to_session_state`.

Logika práce so schémou je umiestnená v triede `SessionManager` v module `core/session_manager.py`. Schéma má podobu JSON dokumentu rozdeleného na bloky (`context`, `auth`, `data` a ďalšie), pričom každý blok obsahuje polia, ktoré počas konverzácie treba vyplniť. Trieda načíta schému zo súboru, zistí, ktoré polia sú zatiaľ prázdne, a podľa poradia definovaného v konštante `BLOCK_ORDER` určí, ktorý blok je na rade. Špecifickú úlohu tu zohrajú polia s prefixom `has_`, ktoré fungujú ako prepínače. Ak je hodnota `auth.has_auth` rovná `false`, znamená to, že aplikácia autentifikáciu nemá a všetky podpolia tohto bloku sa preto vynechajú. Po extrakcii nových faktov ich `SessionManager` zapíše do schémy metódou `merge_updates`. Toto oddelenie zodpovedností umožňuje Orchestrátorovi pracovať len s jednoduchým rozhraním bez znalosti vnútornej štruktúry JSON schém, stačí mu informácia o tom, ktoré pole je potrebné vyplniť ako ďalšie.

SessionRepository a databáza

Modul `core/session_repository.py` obsahuje všetky operácie nad PostgreSQL databázou, ktorá beží v rámci služby Supabase. Modul je navrhnutý podľa vzoru repository, čo znamená, že zvyšok systému komunikuje s databázou len cez funkcie `create_session`, `get_session_by_id`, `add_message`, `update_session` a `delete_session`. Vstupom aj výstupom týchto funkcií sú Pydantic modely, takže volajúci kód neprichádza do styku s SQL dotazmi ani rozhraním Supabase klienta. Ak databáza vráti chybu alebo výsledok dopytu je prázdny, repository vyhodí doménovú výnimku `DatabaseError`, prípadne vráti pripravený fallback objekt. Orchestrátor z toho zistí, že prácu treba prerušiť. Takéto oddelenie má dva praktické dôvody. Funkcie repository sa pri jednotkovom testovaní dajú nahradiť mockom, čím sa zvyšok systému stáva ľahko testovateľným a prípadná výmena databázy by sa premietla iba do jedného modulu. Funkcia `db_to_session_state` prevádza riadky databázovej tabuľky na objekt `SessionState`, s ktorým ďalej pracuje Orchestrátor a funkcia `update_session` realizuje opačný prevod vrátane vygenerovania titulu relácie zo schémy.

Runner

Trieda `Runner` (`agents/runner.py`) predstavuje vrstvu, cez ktorú prechádzajú všetky volania LLM. Hoci každý agent disponuje vlastným `LangChain` objektom `Runnable`, nevolá ho priamo, ale prostredníctvom metódy `Runner.invoke_agent`. Tá obaľuje volanie do bloku s opakovaním. Pri prechodných chybách (zlyhania HTTP, odpovede 5xx zo služby Ollama) sa volanie opakuje s odstupom 1 a 2 sekundy, pričom celkový počet pokusov je obmedzený na tri. Pri trvalých chybách, teda odpovediach 4xx zo služby Ollama alebo pri výnimkách z knižnice `LangChain`, metóda okamžite vyhodí výnimku `AgentExecutionError`, ktorá sa na vyššej vrstve premietla do HTTP odpovede 503. Vďaka centralizácii tejto logiky nie je pri pridávaní nového agenta potrebné opätovne

riešiť odolnosť voči chybám a zlyhania sa prejavujú konzistentne v celom systéme.

RouterService

RouterService (`agents/router_agent.py`) plní v systéme jedinú úlohu, a to zaradiť projekt na základe úvodnej správy používateľa do jednej z piatich kategórií aplikácií. Aktivuje sa iba pri reláciách, ktorých stav ešte nebol určený. Klasifikáciu vykonáva jazykový model pomocou prísne štruktúrovaného promptu, ktorý definuje prípustné kategórie a od modelu vyžaduje odpoveď v podobe jediného slova. Tento reťazec sa následne preloží na zodpovedajúcu hodnotu enumu `AppType`. V prípade, že odpoveď nezodpovedá žiadnej z preddefinovaných kategórií, priradí sa hodnota `OTHER` a riadenie preberá dynamicky generovaný špecialista. Takéto oddelenie umožňuje pre router použiť iný model ako pre konverzáciu, konkrétne taký, ktorý je rýchlejší a deterministickejší.

ValidationService

ValidationService (`agents/validation_agent.py`) má za úlohu získať z prirodzenej reči používateľa štruktúrované údaje, ktoré sa dajú zapísať do schémy. Po každej správe dostane na vstupe aktuálny stav schémy, doterajšiu konverzáciu a zoznam polí, na ktoré sa systém v danej chvíli pýta. Z týchto údajov zostaví prompt s pevne definovaným formátom výstupu (JSON) a s niekoľkými príkladmi, ktoré modelu ukazujú, čo má a čo nemá extrahovať. LLM následne vráti zoznam navrhovaných aktualizácií spolu s hodnotou dôvery. Služba odfiltruje všetky návrhy s dôverou pod 0,8, ďalej tie, ktoré odkazujú na polia, ktoré v schéme neexistujú, a tie, ktorých hodnotu nie je možné previesť na očakávaný typ (boolean, list, string). Príklady v prompte ilustrujú dva podstatné princípy: model nemá uhádnuť údaje, ktoré používateľ nevyslovil, a otázku používateľa nemá interpretovať ako odpoveď. Vďaka tomuto filtru má systém pomerne vysokú mieru istoty, že do schémy zapíše len informácie, ktoré z konverzácie skutočne vyplynuli.

ConversationalAgent

ConversationalAgent (`agents/conversation_agent.py`) je zodpovedný za tvorbu otázok zobrazovaných používateľovi. Na vstupe prijíma doménové pravidlá od špecialistu, aktuálnu schému, zoznam chýbajúcich polí spolu s ich popismi, históriu konverzácie a poslednú správu od používateľa. Prompt definuje viacero obmedzení, agent kladie naraz najviac dve otázky, môže sa pýtať len na polia uvedené v zozname chýbajúcich polí a nesmie sa pýtať na to, čo už používateľ v histórii raz zodpovedal. Ak používateľ namiesto odpovede žiada vysvetlenie alebo príklad, agent mu vysvetlenie poskytne a otázku položí neskôr.

ChecklistGenerator

Komponent ChecklistGenerator (`agents/checklist_generator.py`) predstavuje záverečnú fázu, ktorá sa aktivuje, keď schéma neobsahuje žiadne nevyplnené polia. V prvom kroku načíta rizikové pravidlá aktívneho špecialistu (`risk_rules`) a ponechá z

nich len tie, ktorých podmienky sú voči aktuálnemu stavu schémy splnené. Vyfiltrované riziká následne usporiada podľa štruktúry `priority_map`, ktorá určuje poradie blokov v závislosti od typu aplikácie (napríklad bloky `auth` a `data` sa vyhodnocujú skôr než `ui`). Pri zhode v poradí blokov rozhoduje priorita uvedená priamo v pravidle (HIGH, MEDIUM, LOW). Výsledkom je textový checklist s pevne stanovenou štruktúrou troch prioritných skupín, ktorý je možné v ďalšej fáze exportovať.

Špecialisti

Trieda obsahujúca doménové znalosti pre konkrétny typ aplikácie sa v práci označuje ako

špecialistom. Všetky takéto triedy dedia od abstraktnej triedy `BaseSpecialistAgent`, ktorá vymedzuje spoločné rozhranie. To pozostáva z poľa `schema_file` s názvom súboru obsahujúceho základnú JSON schému, mapy `hints_map`, ktorá ceste k poľu priradzuje jeho ľudsky čitateľný popis, textového atribútu `domain_rules` obsahujúceho krátky pokyn pre konverzačného agenta a dvojice metód `get_risk_rules` a `get_priority_map`, ktoré využíva `ChecklistGenerator`. Špecialista je teda iba konfiguráciou, neobsahuje vlastnú logiku, len dáta. Tento prístup uľahčuje rozšírenie systému o ďalšie typy aplikácií.

WebAgent má spomedzi všetkých špecialistov najrozsiahlejšie spracovanú schému. Tá pokrýva oblasti, ktoré sa typicky vyskytujú vo webových aplikáciách: kontext projektu, používateľské rozhranie, autentifikáciu, prácu s dátami, integrácie, nefunkčné požiadavky a prostredie. Súčasťou schémy je aj rozsiahla sada rizikových pravidiel zameraných na bezpečnostné hrozby (XSS, SQL injection, narušená autentifikácia), validáciu vstupov, nahrávanie súborov a spoľahlivosť integrácií.

MobileAgent pokrýva oblasti špecifické pre mobilné aplikácie: cieľové platformy (Android, iOS), oprávnenia zariadenia, biometrickú autentifikáciu, offline režim a synchronizáciu lokálneho úložiska.

DesktopAgent pokrýva oblasť inštalácie, prepojenia s operačným systémom (automatické aktualizácie, registry, súborový systém) a osobitosti spojené s nasadením desktopovej aplikácie.

APIAgent je určený pre backendové služby bez používateľského rozhrania. Medzi jeho bloky patrí typ rozhrania (REST, GraphQL, gRPC), autentifikácia pomocou tokenov, definícia endpointov, formát chybových odpovedí, verzionovanie a rate limiting. Samostatnú časť tvoria bezpečnostné požiadavky, ktoré sú pri API obzvlášť dôležité vzhľadom na priame vystavenie rozhrania klientom.

OtherAgent je zámerne navrhnutý ako minimalistický agent. Použije sa v prípade, keď je hodnota `AppType` nastavená na `UNKNOWN`, prípadne pri typoch aplikácií, ktoré nespádajú pod žiadnu z predchádzajúcich kategórií. Jeho schéma nie je definovaná staticky, ale vyplňa sa za behu prostredníctvom triedy `SpecialistGenerator`, ktorá agentovi priradí atribúty `hints_map` a `domain_rules`.

Risk rules a Priority map

Súčasťou každého špecialistu sú dve dátové štruktúry, `risk_rules` a `priority_map`, ktoré slúžia ako prepojenie medzi vyplnenou schémou a finálnym checklistom.

`Risk_rules` je zoznam pravidiel, pričom každé pravidlo obsahuje podmienky vo forme polí v schéme, názov rizika, kategóriu (`security`, `validation`, `performance` a podobne), prioritu a stručné odôvodnenie. Pravidlo sa uplatní vtedy, keď všetky cesty uvedené v jeho podmienkach majú v schéme pravdivú hodnotu. Pravidlo `sql_injection` sa napríklad aktivuje iba vtedy, keď aplikácia obsahuje formuláre aj databázu, čiže sú splnené podmienky `data.has_forms` a `data.has_database`. Tento prístup má dve výhody. ChecklistGenerator nepotrebuje žiadnu doménovú logiku a vystačí si s vyhodnotením podmienok, pričom samotné pravidlá je možné upravovať bez zásahu do kódu agenta.

Štruktúra `priority_map` priradzuje jednotlivým prioritám (`HIGH`, `MEDIUM`, `LOW`) zoznamy blokov schémy. Generátor na jej základe zoradzuje aplikovateľné riziká tak, aby riziká pochádzajúce z bezpečnostne citlivejších blokov (predovšetkým `auth` a `data`) boli v testovacom checkliste umiestnené pred rizikami z blokov s nižšou mierou kritickosti (napríklad `ui` alebo `context`). Bez tejto mapy by bolo poradie testov v checkliste náhodné a používateľ by si dôležité kontroly musel vyhľadávať sám. Tieto dve štruktúry teda spolu zabezpečujú, že prístup k testovaniu je pre každý typ aplikácie definovaný formou dát, čím odpadá potreba zasahovať za týmto účelom do zdrojového kódu.

SpecialistGenerator

SpecialistGenerator (`agents/specialist_generator.py`) sa používa v prípadoch, keď Router vyhodnotí typ aplikácie ako `OTHER` alebo keď používateľ opíše projekt, pre ktorý zatiaľ nie je k dispozícii pripravená schéma. V takejto situácii vopred nie je známe, ktoré bloky a polia má používateľ vyplniť, a preto ich musíme vygenerovať až počas behu. Generátor postupne zavolá troch podagentov a ich výstupy spojí do objektu `SpecialistDefinition`, ktorý Orchestrator následne odovzdá agentovi `OtherAgent`. SpecialistGenerator sám o sebe zabezpečuje len riadenie týchto troch krokov.

Komponent SchemaGenerator dostáva na vstupe používateľský opis projektu a existujúce schémy, ktoré preňho predstavujú referenčné vzory. Na ich základe vygeneruje novú JSON schému s rovnakou štruktúrou, teda s blokom `context`, jeho podblokmi a prepínačmi `has_*`, pričom polia prispôbi konkrétnemu typu projektu. Keďže model pracuje s tvarovou šablónou existujúcich schém, výsledná schéma je priamo kompatibilná so zvyškom systému a nevyžaduje dodatočné úpravy.

Trieda HintsGenerator vytvára pre novú schému štruktúru `hints_map`, teda slovník, v ktorom je každej ceste v schéme priradený krátky textový popis daného poľa. Tieto popisy potom využíva agent ConversationalAgent pri formulovaní otázok na nevyplnené polia. Pri generovaní vychádza HintsGenerator z existujúcich hint máp všetkých

štyroch zabudovaných špecialistov, vďaka čomu si výstup zachováva rovnaký štýl ako ručne písané popisy.

DomainRulesGenerator vytvorí krátky pokyn v rozsahu troch až šiestich viet, ktorý opisuje, čím sa daný projekt zaoberá a aké otázky by mal konverzačný agent klásť prednostne. Tento pokyn sa neskôr vkladá na začiatok promptu pre ConversationalAgent a slúži mu ako doménový kontext pri vedení rozhovoru.

Export do Excelu

Posledná časť backendu zabezpečuje export hotového checklistu do súboru vo formáte XLSX. Po stlačení tlačidla na export vo frontende sa zavolá endpoint

GET /sessions/{id}/export. Endpoint prejde históriou konverzácie a vyhľadá poslednú správu od asistenta, ktorá obsahuje checklist, rozpozná ju podľa hlavičiek [HIGH PRIORITY], [MEDIUM PRIORITY] a [LOW PRIORITY].

Text tejto správy sa odovzdá funkcii `parse_checklist`. Funkcia pomocou regulárnych výrazov rozdelí obsah na dvojice v tvare (priorita, testovací prípad) a vráti ich funkcii `generate_excel`. Tá s využitím knižnice `openpyxl` vytvorí pracovný hárok s dvoma stĺpcami, kde prvý obsahuje prioritu a druhý popis testu. Riadky sa farebne odlišia podľa priority: vysoká priorita je označená červenou, stredná žltou a nízka zelenou. Hotový súbor sa vráti klientovi ako `StreamingResponse`.

Pri návrhu bol export oddelený od samotného generovania checklistu. Parser nevyvoláva jazykový model a modul pracujúci s knižnicou `openpyxl` nepozná štruktúru konverzácie. Vďaka tomu je možné obe časti vyvíjať a testovať nezávisle od seba.

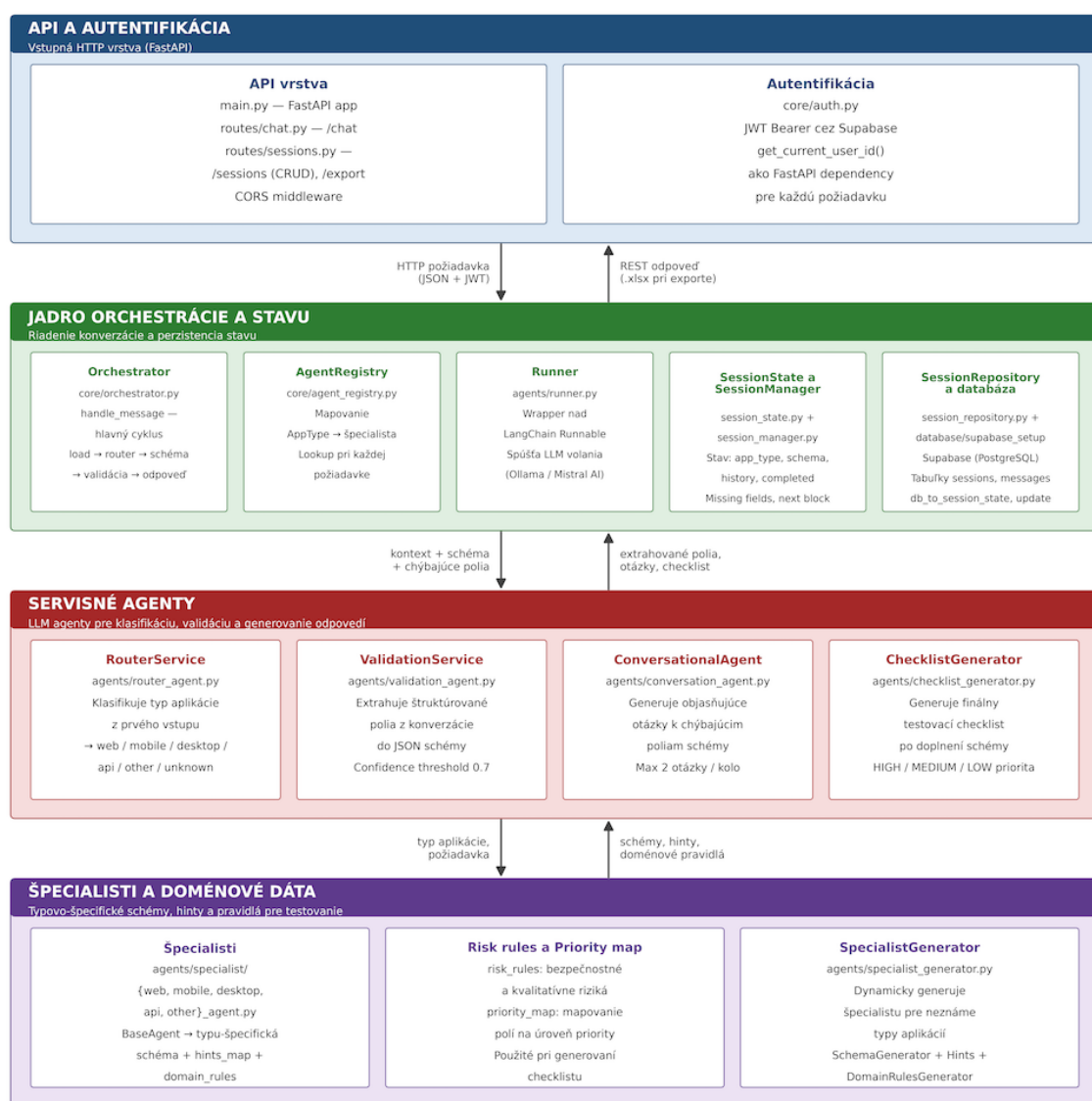
Súhrnný pohľad na architektúru

Vzájomné usporiadanie opísaných komponentov je znázornené na obrázku 5.1. Systém je rozdelený do štyroch logických vrstiev. Najvyššiu vrstvu tvorí *API a autentifikácia*, vstupná HTTP brána postavená na frameworku FastAPI, ktorá overuje JWT tokeny prostredníctvom služby Supabase a smeruje požiadavky do nižších vrstiev. Pod ňou sa nachádza *jadro orchestrácie a stavu*, ktoré riadi priebeh konverzácie a zabezpečuje perzistenciu. Orchestrátor koordinuje ostatné komponenty, AgentRegistry slúži na výber špecialistu, Runner obaľuje volania jazykového modelu a SessionState spolu so SessionRepository udržiavajú stav relácie medzi jednotlivými HTTP požiadavkami.

Tretiu vrstvu tvoria *servisné agenty*, ktoré priamo komunikujú s jazykovým modelom: RouterService klasifikuje typ aplikácie, ValidationService extrahuje štruktúrované dáta z prirodzenej reči, ConversationalAgent generuje otázky pre používateľa a ChecklistGenerator zostavuje výsledný checklist. Najnižšia vrstva (*špecialisti a doménové dáta*) obsahuje typovo špecifické schémy, hint mapy a rizikové pravidlá. Pri neznámych typoch aplikácií sa tieto dáta dynamicky generujú prostredníctvom komponentu

SpecialistGenerator.

Diagram zachytáva aj smerovanie dát medzi vrstvami. Z API vrstvy do jadra prúdia HTTP požiadavky vo formáte JSON spolu s JWT tokenom, opačným smerom REST odpovede, prípadne XLSX súbor pri exporte. Medzi jadrom a servisnými agentmi sa odovzdáva kontext relácie, schéma a zoznam chýbajúcich polí. Spätne prichádzajú extrahované polia, otázky alebo hotový checklist. Servisní agenti čerpajú zo špecialistov schémy, hint mapy a doménové pravidlá, ktoré používajú pri formulovaní otázok aj pri vyhodnocovaní rizikových pravidiel. Toto vrstvom usporiadanie zabezpečuje, že každá vrstva má jasne definovanú zodpovednosť a komunikuje len so susednými vrstvami.



Obr. 5.1: Architektúra systému - vrstvom usporiadanie komponentov a tok dát medzi nimi.

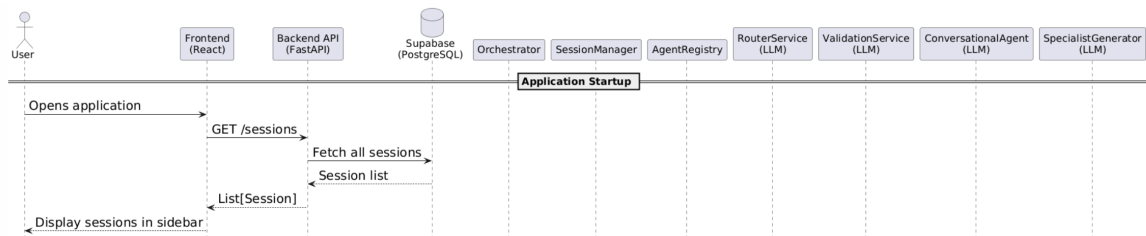
5.2 Priebeh spracovania požiadavky

Nasledujúce podkapitoly opisujú, ako medzi sebou komunikujú jednotlivé komponenty počas typického používania aplikácie. Každú fázu dopĺňa časť sekvenčného diagramu, ktorý daný tok ilustruje.

5.2.1 Začiatok aplikácie

Spustenie aplikácie je prvou fázou, ku ktorej dochádza, keď prihlásený používateľ otvorí webové rozhranie chatbota. Aplikácia v tomto momente načíta zoznam existujúcich konverzácií používateľa, aby si mohol prezrieť priebeh prechádzajúcich relácií alebo začať novú.

Otvorenie aplikácie automaticky spustí požiadavku `GET /sessions` z frontendu na backendové API. Backend ju spracuje, opýta sa databázy na uložené relácie používateľa, databáza vráti zoznam relácií a backend ho zabalí do odpovede typu `List[Session]` a pošle späť na frontend. Frontend zoznam relácií následne zobrazí v bočnom paneli, pričom dokončené konverzácie odliší zelenou farbou. Tento priebeh je znázornený na obrázku 5.2.



Obr. 5.2: Sekvenčný diagram fázy Začiatok aplikácie.

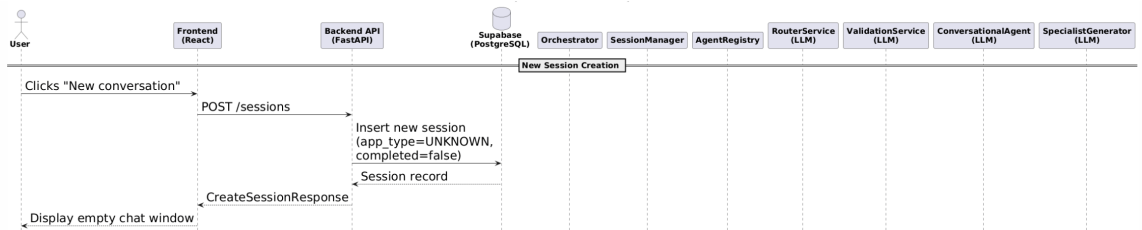
Po dokončení tejto fázy je aplikácia pripravená na ďalšiu interakciu, buď na prezretie existujúcej konverzácie, alebo na vytvorenie novej.

5.2.2 Vytvorenie novej relácie

Nová relácia vznikne po kliknutí na položku "Nová konverzácia" v bočnom paneli. Cieľom tejto fázy je uložiť v databáze nový záznam relácie, do ktorého budú postupne ukladať všetky správy z rozhovoru.

Po stlačení tlačidla frontend odošle požiadavku `POST /sessions` na backendové API. Backend na jej základe vytvorí v databáze nový záznam relácie s predvolenými

hodnotami: typ aplikácie dostane hodnotu `UNKNOWN`, pretože v tomto momente ešte nie je známe, o aký druh aplikácie ide, a stav dokončenia sa nastaví na `false`. Novovytvorený záznam databáza odošle späť backendu, ktorý ho vloží do typu `CreateSessionResponse` a doručí frontendu. Tento priebeh je zobrazený na 5.3.



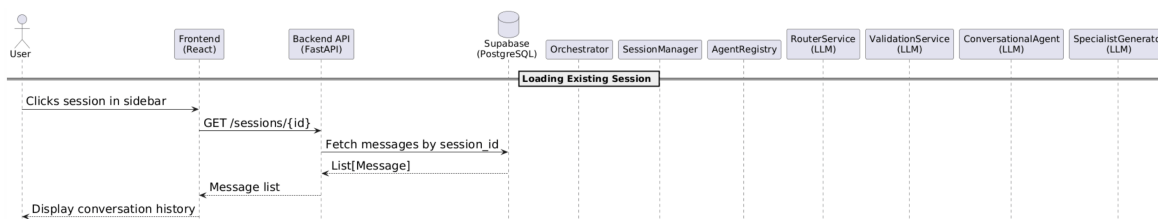
Obr. 5.3: Sekvenčný diagram fázy Vytvorenie novej relácie.

Po dokončení tejto fázy môže používateľ začať komunikovať s chatbotom.

5.2.3 Načítanie existujúcej relácie

Po kliknutí na niektorú z predchádzajúcich konverzácií aplikácia načíta zodpovedajúcu reláciu. Táto fáza umožňuje používateľovi zobraziť kompletný obsah zvolenej konverzácie.

Frontend po kliknutí odošle požiadavku `GET /sessionid` na backendové API, pričom `id` je identifikátor vybranej relácie. Backend na základe identifikátora získa z databázy všetky správy patriace danej relácii a odošle ich späť frontendu ako `List[Message]`. Frontend ich zobrazí v hlavnom okne chatu v poradí, v akom vznikli, čím používateľovi sprístupní celú históriu danej konverzácie. Tento krok je zobrazený na obrázku 5.4.



Obr. 5.4: Sekvenčný diagram fázy Načítanie existujúcej relácie.

5.2.4 Prvá správa a detekcia typu aplikácie

Po vytvorení novej relácie nasleduje fáza, v ktorej sa odošle prvá správa od používateľa. Na základe jej obsahu sa identifikuje typ aplikácie a systém si pripraví zodpovedajúcu sadu otázok pre nasledujúcu konverzáciu.

Pri otvorení prázdneho okna chatu má používateľ dve možnosti, ako pokračovať. Buď napíše vlastnú správu s opisom svojho projektu, alebo klikne na jedno zo štyroch predpripravených tlačidiel: *I'm building a web app*, *I'm building a mobile app*, *I'm building a desktop app* a *I'm building an API*. Tieto tlačidlá zodpovedajú štyrom hlavným typom aplikácií, ktoré chatbot pozná, a umožňujú rýchly štart konverzácie bez potreby formulovať úvodnú správu od nuly.

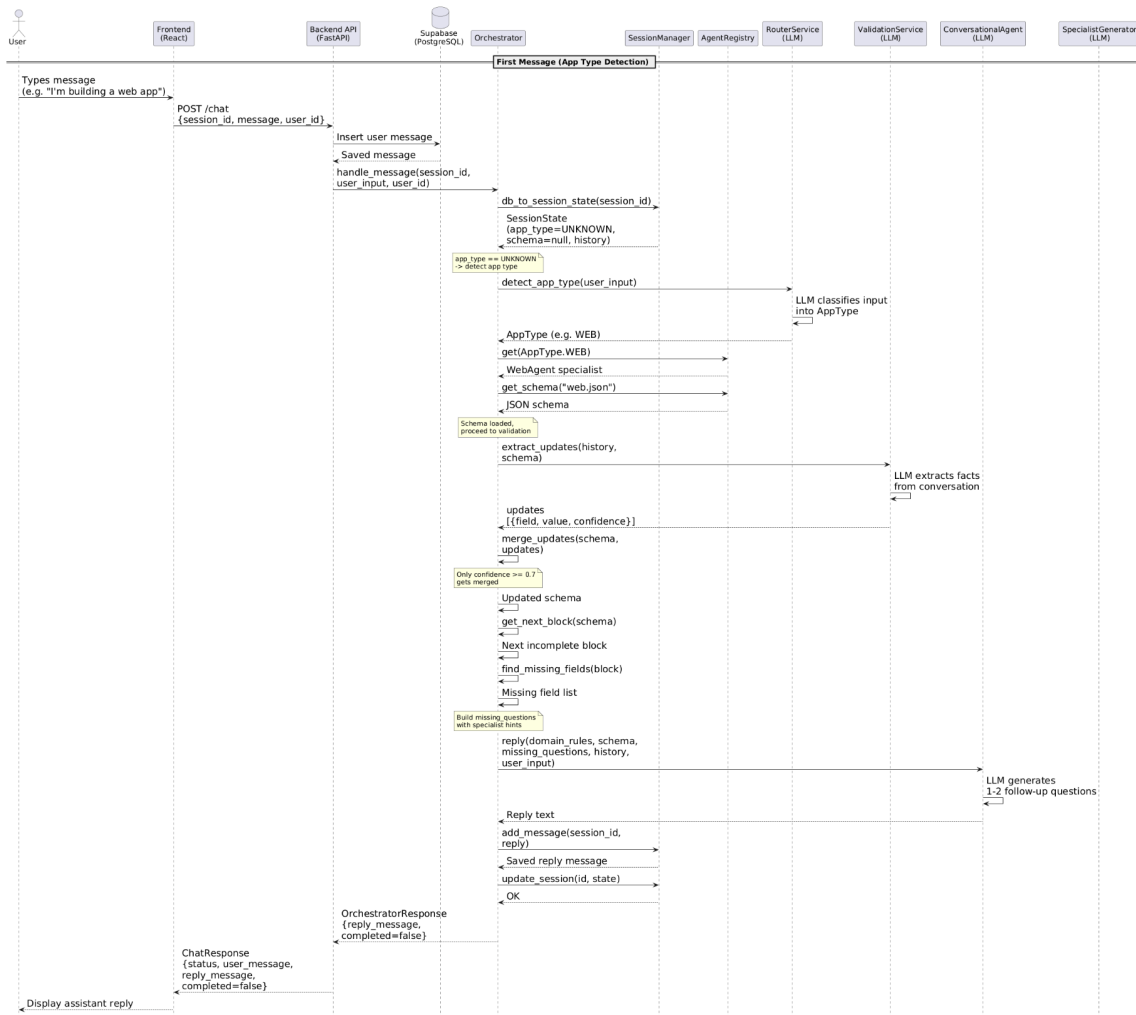
Po odoslaní správy frontend pošle požiadavku `POST /chat` s identifikátorom relácie a textom správy. Backend z JWT tokenu získa identifikátor používateľa a overí, či mu daná relácia patrí. Správu následne uloží do databázy a odovzdá Orchestrátorovi, ktorý riadi priebeh konverzácie.

Orchestrátor začína načítaním aktuálneho stavu z databázy. Keďže typ aplikácie zatiaľ nie je určený, správa používateľa sa odovzdá službe `RouterService`. Tá zaradí správu pomocou jazykového modelu do niektorej z podporovaných kategórií, napríklad `WEB`. Na základe tohto zaradenia si Orchestrátor vyžiada od `AgentRegistry` príslušného špecialistu (v tomto prípade `WebAgent`) a od neho načíta zodpovedajúcu schému, ktorá definuje, aké projektové informácie sú potrebné na vytvorenie checklistu.

Po načítaní schémy nasleduje validácia. Orchestrátor odovzdá riadenie službe `ValidationService`, ktorá z histórie konverzácie extrahuje získané fakty spolu s hodnotou istoty (*confidence*). Do schémy sa zapisujú len tie informácie, ktorých miera istoty je dostatočne vysoká, čím sa predchádza ukladaniu nepresných údajov.

Po aktualizácii schémy Orchestrátor nájde prvý nedokončený blok a zostaví zoznam polí, ktoré v ňom ešte chýbajú. K týmto poliam priradí pomocné nápovedy od špecialistu pre príslušný typ aplikácie a takto pripravené podklady odovzdá konverzačnému agentovi (`ConversationalAgent`). Ten na základe schémy, doménových pravidiel, histórie konverzácie a aktuálneho vstupu pripraví jednu až dve otázky pre používateľa.

Nová správa sa zapíše do databázy spolu s aktualizovaným stavom relácie a Orchestrátor vráti odpoveď backendu s hodnotou `completed = false`, čo znamená, že konverzácia pokračuje. Backend odpoveď vloží do štruktúry `ChatResponse` a odošle ju frontendu na zobrazenie. Priebeh procesu je zachytený na obrázku 5.5.



Obr. 5.5: Sekvenčný diagram fázy Prvá správa a detekcia typu aplikácie.

Po dokončení tejto fázy je relácia pripravená na ďalšie kolá otázok a odpovedí, počas ktorých sa schéma postupne dopĺňa.

5.2.5 Cyklus dopĺňania schémy

Po identifikácii typu aplikácie sa začína hlavná fáza rozhovoru, počas ktorej chatbot získava od používateľa informácie pole po poli. Cyklus pokračuje, kým schéma nie je kompletne vyplnená.

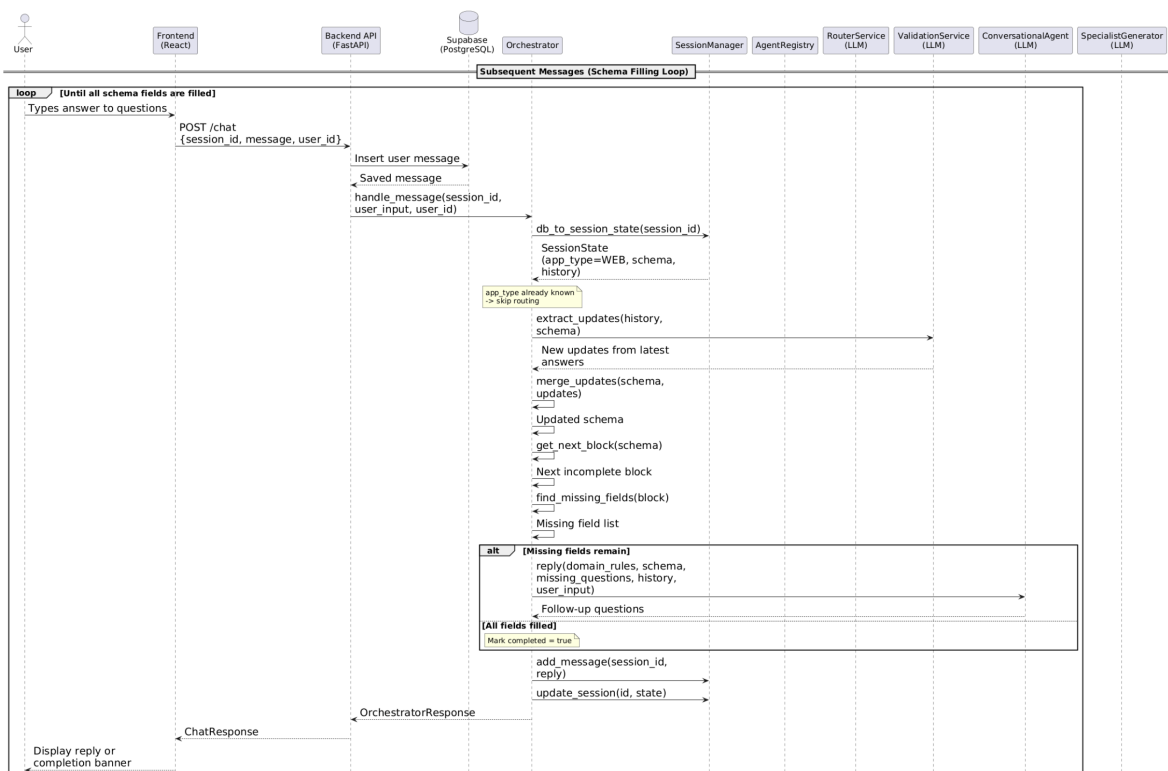
Ďalšie správy od používateľa idú rovnakou cestou ako tá úvodná: frontend ich pošle cez `POST /chat`, backend ich uloží a odovzdá Orchestrátorovi. Orchestrátor načíta aktuálny stav relácie. Krok zisťovania typu aplikácie sa však vynechá, keďže typ bol do stavu relácie zapísaný už v prvom kole.

Orchestrátor zavolá `ValidationService`, ktorá z histórie dialógu získa nové informácie o projekte obsiahnuté v posledných správach od používateľa. Po overení ich spoľah-

livosti sa tieto informácie zaznamenajú do schémy. Orchestrátor potom skontroluje, ktorá časť schémy ešte nie je úplná, a zostaví zoznam chýbajúcich údajov.

V závislosti od stavu schémy sa konverzácia v tomto kroku ubera jedným z dvoch smerov. Ak v schéme ostávajú nevyplnené polia, Orchestrátor požiada ConversationalAgent o ďalšiu otázku alebo dvojicu otázok zameranú na ich doplnenie. Keď sú všetky informácie zhromaždené, relácia dostane označenie dokončená prostredníctvom príznaku `completed = true`.

Výsledná odpoveď, prípadne správa o ukončení, sa zapíše do databázy spolu s aktualizovaným stavom relácie. Orchestrátor odovzdá odpoveď backendu, ktorý ju zašle frontendu v štruktúre `ChatResponse`. Frontend na základe hodnoty príznaku `completed` rozhodne, či zobrazí správu od chatbota, alebo informačný panel s oznámením, že checklist sa vygeneroval. Celý tento cyklus je zobrazený na obrázku 5.6.



Obr. 5.6: Sekvenčný diagram fázy Cyklus dopĺňania schémy.

Po označení relácie ako dokončenej je testovací checklist pripravený na export alebo na prezretie používateľom.

5.2.6 Dynamické generovanie špecialistu

V prechádzajúcich podkapitolách bol opísaný priebeh konverzačného toku pre štyri konkrétne kategórie projektov: web, mobile, desktop a API. Nie každý projekt však do niektorej z nich zapadá, napríklad nástroje typu CLI. Pre tieto prípady systém disponuje mechanizmom dynamického generovania špecialistu.

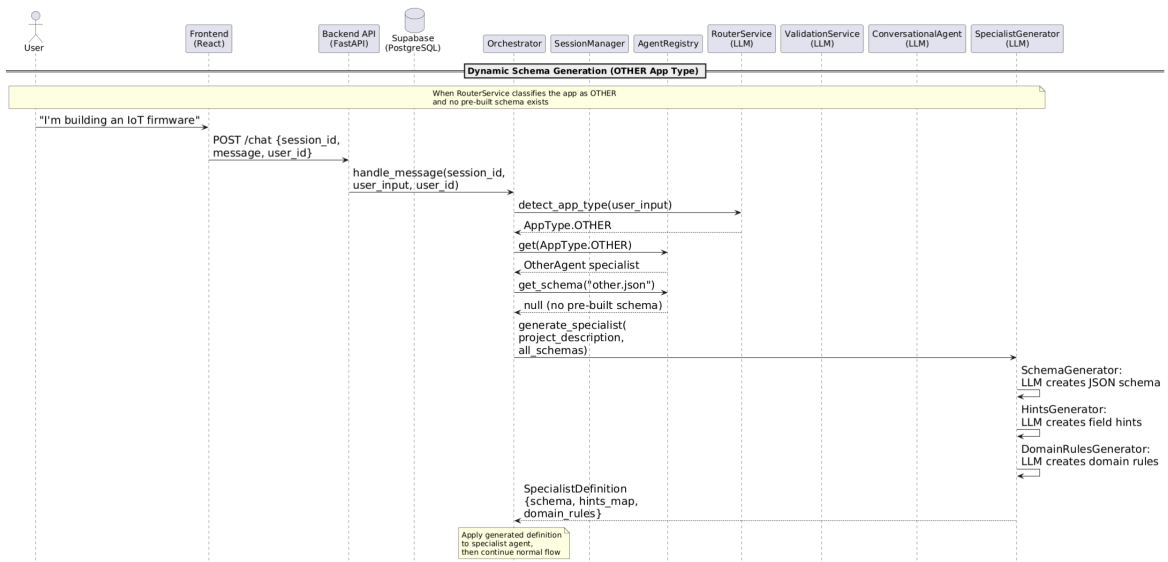
Komunikácia začína rovnako ako pri spracovaní prvej správy. Frontend odošle požiadavku na `POST /chat`, backend ju uloží a odovzdá Orchestrátorovi. Ten zavolá službu `RouterService`, ktorá ju zaradí do kategórie `OTHER`, čo signalizuje, že daný typ aplikácie nespadá ani do jednej z preddefinovaných kategórií.

Orchestrátor si následne vyžiada od registra agentov špecialistu pre kategóriu `OTHER` (`OtherAgent`) a pokúsi sa od neho načítať vopred definovanú JSON schému. Keďže taká schéma pre kategóriu neexistuje, `OtherAgent` vráti hodnotu `None`.

V tomto bode vstupuje do procesu komponent `SpecialistGenerator`, ktorý zodpovedá za vytvorenie špecialistu zodpovedajúceho danému projektu používateľa. Metóda `generate_specialist` dostane na vstupe popis projektu spolu so všetkými existujúcimi schémami, ktoré jazykovému modelu poskytujú referenciu štýlu. Komponent vnútorne aktivuje tri samostatné podkomponenty:

- `SchemaGenerator` vytvorí novú JSON schému prispôbenú danému typu projektu
- `HintsGenerator` vygeneruje pomocné nápovedy k jednotlivým poliam schémy
- `DomainRulesGenerator` zostaví súbor doménových pravidiel, ktoré sa neskôr odovzdajú konverzačnému agentovi.

Výsledkom tohto procesu je objekt `SpecialistDefinition` obsahujúci schému, nápovedy a doménové pravidlá. Orchestrátor prideli túto definíciu príslušnému špecialistovi a od tohto momentu pokračuje konverzácia rovnakým spôsobom ako pri preddefinovaných typoch aplikácií, teda štandardným cyklom dopĺňania schémy. Tento priebeh je znázornený na obrázku 5.7.



Obr. 5.7: Sekvenčný diagram fázy Dynamické generovanie špecialistu.

Vďaka tomuto mechanizmu dokáže chatbot pomôcť aj s projektmi, na ktoré nebol vopred pripravený, bez potreby manuálne dopĺňať novú schému do kódu aplikácie.

Tým je opísaný kompletný tok komunikácie systému, od otvorenia aplikácie po dokončenie konverzácie a vygenerovanie testovacieho checklistu. Z opísaných fáz vyplývajú dve vlastnosti návrhu. Prvou je centrálna úloha Orchestrátora, ktorý ako jediný riadi priebeh konverzácie a koordinuje ostatné komponenty (RouterService, ValidationService, špecialistov a ConversationalAgent). Druhou je oddelenie dát od logiky. Stav každej relácie sa uchováva v databáze a backend si ho pri každej prichádzajúcej správe nanovo načíta, takže aplikácia nemusí udržiavať konverzačný stav v pamäti medzi jednotlivými HTTP požiadavkami.

5.3 Diagram aktivity systému

Diagram aktivít (obr.5.8) zachytáva celý priebeh interakcie so systémom. Od momentu spustenia aplikácie až po okamih, keď je testovací checklist hotový. Celý proces je rozdelený do piatich plavečkových dráh, ktoré zodpovedajú jednotlivým komponentom systému.

Inicializácia a správa relácií Pri spustení aplikácie frontend okamžite odošle požiadavku GET /sessions a výsledný zoznam existujúcich relácií zobrazí v bočnom paneli. Používateľ si následne môže vybrať jednu z dostupných relácií, frontend vtedy zavolá GET /session/{id} a databáza vráti celú históriu danej konverzácie, alebo môže začať

zaradenia sa potom z **AgentRegistry** načíta príslušný špecializovaný agent, ktorý sa postará o ďalší priebeh konverzácie.

Schéma checklistu Pre typy Web, Mobile, Desktop a API je predpripravená JSON schéma uložená v súbore, ktorá určuje bloky a polia checklistu pre daný typ aplikácie. Pre typ Other predpripravená schéma neexistuje. Namiesto nej ju zostavujú traja agenti dynamicky. **SchemaGenerator** pripraví základnú štruktúru, **HintsGenerator** doplní k poliam popisky a nápovedy, z ktorých čerpá konverzačný agent počas rozhovoru, a **DomainRulesGenerator** definuje pravidlá zodpovedajúce konkrétnej doméne projektu. Takto zostrojená **SpecialistDefinition** vstupuje do ďalšieho spracovania rovnakou cestou ako ktorákoľvek predpripravená schéma.

Validácia a extrakcia faktov **ValidationService** prechádza históriou konverzácie a extrahuje z nej fakty relevantné pre daný projekt. Každému extrahovanému faktu sa priradí skóre spoľahlivosti a do schémy sa zaradia len tie, ktoré dosiahnú hodnotu aspoň 0,8. Všetky fakty, ktoré touto kontrolou prejdú, sa na záver zlúčia s aktuálnym stavom schémy.

Generovanie otázok alebo dokončenie Orchestrátor prechádza schému blok po bloku a keď narazí na ďalší nekompletný úsek, prejde si jeho polia a určí, čo v ňom chýba. Z nápoved špecialistu pre daný typ aplikácie následne poskladá otázky, ktoré má položiť používateľovi.

V prípade, že niektoré polia zostanú nevyplnené, **ConversationalAgent** na ich základe sformuluje jednu alebo dve ciele otázky a predloží ich používateľovi. Keď sú všetky požadované polia riadne vyplnené, systém reláciu uzavrie a automaticky vygeneruje záverečnú správu.

Uloženie odpovede a aktualizácia stavu Vygenerovaná odpoveď asistenta sa zapíše do databázy. Orchestrátor následne aktualizuje uložený stav relácie, konkrétne schému, typ aplikácie a príznak dokončenia. Backend potom odošle frontendu objekt **ChatResponse**, ktorý obsahuje text pôvodnej správy používateľa, odpoveď asistenta a informáciu o tom, či bola konverzácia dokončená.

Správy z frontendu sa ihneď zobrazia v okne chatu. Ak konverzácia ešte neskončila, používateľ napíše ďalšiu správu a celý postup sa zopakuje. Takto to pokračuje dovtedy, kým nie sú vyplnené všetky požadované polia. Po ich vyplnení sa v rozhraní objaví informačná lišta a pole na zadávanie textu sa zablokuje.

Kapitola 6

Overenie a zhodnotenie riešenia

Po implementácii systému bolo potrebné overiť, či navrhnuté riešenie funguje tak, ako sa od neho očakáva, a či vygenerovaný checklist predstavuje zmysluplný podklad pre testovanie. Overovanie malo formu používateľského testovania s externým respondentom (sekcia 6.3). Systém som primárne testoval ja ako autor práce, pričom som pripravil niekoľko opisov projektov pokrývajúcich všetky podporované typy aplikácií a postupne som ich prešiel.

6.1 Spôsob overenia riešenia

Overovanie som rozdelil do troch oblastí. Prvou bolo správanie chatbota pre štyri preddefinované typy aplikácií: webovú, mobilnú, desktopovú a API službu. Pre každý typ som pripravil opis projektu a sledoval som, či RouterService správne zaradí projekt do očakávanej kategórie a či následný priebeh konverzácie zodpovedá schéme príslušného špecialistu.

Druhou oblasťou bolo dynamické generovanie špecialistu cez OtherAgent. Tento mechanizmus sa aktivuje, keď projekt nespadá do žiadnej z preddefinovaných kategórií. Skúšal som ho na opise CLI nástroja, ktorý do žiadnej zo štyroch hlavných kategórií zámerne nepatrí. Sledoval som, či SchemaGenerator vytvorí použiteľnú schému, či HintsGenerator doplní rozumné popisy polí a či DomainRulesGenerator pripraví pokyn, ktorý ConversationalAgent následne pri konverzácii uplatní.

Tretia oblasť sa venovala kvalite výstupného checklistu. Pre každú konverzáciu som vyhodnotil, či vygenerovaný zoznam obsahuje testovacie scenáre relevantné pre opísaný projekt, či sa v ňom nenachádzajú položky pre vlastnosti, ktoré používateľ označil ako neaktuálne, a či prioritizácia zodpovedá rizikovému profilu projektu. Súčasťou tejto oblasti bolo aj overenie exportu do XLSX súboru, teda či parser správne rozpozná hlavičky priorít a či výsledný hárok zodpovedá obsahu konverzácie.

Všetky scenáre prebiehali na lokálnom modeli spúšťanom cez Ollama. Pri overovaní

som použil rovnakú konfiguráciu modelov, aká je predvolená pre produkčné nasadenie, aby výsledky pozorovania zodpovedali tomu, čo dostane bežný používateľ.

6.2 Príklady použitia

V tejto sekcii uvediem dva podrobné príklady použitia systému. Prvý sa týka webovej aplikácie ako reprezentanta preddefinovaných typov a druhý sa zameriava na CLI nástroj, na ktorom systém demonštruje dynamické generovanie špecialistu prostredníctvom OtherAgentu. Konverzácie pre mobilnú, desktopovú a API aplikáciu prebiehajú podľa rovnakého vzorca ako pri webovej aplikácii a líšia sa len schémou príslušného špecialistu.

6.2.1 Webová aplikácia: TaskFlow

TaskFlow je hypotetický projekt pripravený na overenie celého toku konverzácie pre webovú aplikáciu. Ide o aplikáciu pre študentov na manažment času, ktorá je verejne dostupná, podporuje dva režimy zobrazenia a funguje vo vybraných prehliadačoch. Aplikácia má autentifikáciu, dve používateľské role, formuláre, prácu so súborami, REST API, integrácie s tretími stranami a relatívne vysoké výkonnostné nároky. Pre overenie systému to bola vhodná voľba, pretože pokrýva väčšinu blokov definovaných v schéme WebAgentu.

Priebeh konverzácie

Po vytvorení novej relácie bola zvolená predpripravená voľba *I'm building a web app*, čím bola služba RouterService usmernená priamo na kategóriu WEB. Konverzácia pokračovala otázkami, ktoré ConversationalAgent generoval na základe nevyplnených polí v schéme. Skrátený prepis konverzácie je v ukážke 6.1.

Používateľ: I'm building a web app

Chatbot: What's the name of your web app, and what does it do?

Používateľ: My app is called TaskFlow. It's a time management app for students. It's a public app.

Chatbot: Who are the main users of TaskFlow?

Používateľ: students

Chatbot: Does TaskFlow need to support dark mode?

Používateľ: The UI is responsive and has animations. Supported browsers are Firefox and Safari. Supported devices are phones and tablets. It has dark mode and requirements including screen reader support.

Chatbot: Does TaskFlow need users to log in, and if so, what authentication methods should it support (for example email/password, Google, or school SSO)?

Používateľ: The app has authentication via email/password and Google OAuth. There are roles: student and admin. There is an admin panel. Sessions expire after there is a remember me option, and concurrent sessions are limited to one.

Chatbot: Does TaskFlow have a search feature for finding tasks or content?

Používateľ: yes

Chatbot: Does TaskFlow need a database to store users, tasks, or progress?

Používateľ: The app has forms. It uses a database. It supports file upload and file download. Users can create their own content. There is a search feature and pagination.

Chatbot: Does TaskFlow need to communicate with an API, and if so, what type should it use, such as REST or GraphQL?

Používateľ: The app has a REST API. It has payments. It sends email notifications. It integrates with third-party services: Google Calendar, Stripe, and SendGrid.

Chatbot: Does TaskFlow need analytics tracking, error monitoring, SEO optimization, or specific performance targets?

Používateľ: There are performance requirements with up to 10000 concurrent users expected. There are SEO requirements. There is analytics and error monitoring. There are multiple environments: development, staging, and production. There are feature flags. The app is localized and supports English, Slovak, and German.

Z konverzácie sú zrejmé dve vlastnosti, ktoré boli pri overovaní sledované. Po prvé, používateľ často odpovedal naraz na viacero pripravených otázok. Konkrétne pri otázke o tmavom režime používateľ doplnil aj informácie o prehliadačoch, podporovaných zariadeniach a požiadavkách na čítačku obrazovky, hoci sa systém pýtal len na tmavý režim. ValidationService tieto údaje extrahoval správne a pri ďalšej iterácii sa už chatbot na ne nepýtal. Po druhé, podmienené bloky fungovali tak, ako bolo navrhnuté. Keď používateľ uviedol, že aplikácia podporuje vyhľadávanie, chatbot pokračoval otázkami o databáze a stránkovaní, ale na vyhľadávanie samotné sa už nepýtal.

Vygenerovaný checklist

Po vyplnení všetkých povinných polí schémy systém vygeneroval checklist rozdelený na prioritné úrovne. Sekcia s vysokou prioritou je zobrazená nižšie a obsahuje rizikové pravidlá, ktoré sa aktivovali pre TaskFlow.

[HIGH PRIORITY]

- Verify that authentication using email/password enforces correct credentials and properly handles failed login attempts (TaskFlow student/admin roles).
- Test Google OAuth login flow, ensuring tokens are validated and user account is created/linked appropriately.
- Check that session tokens expire after the configured timeout, and users are logged out and redirected to the login page.
- Attempt to access the admin panel as a student and as an unauthenticated user, verify access is denied and no privileged actions are exposed.
- Confirm that admin-only functionality and API routes are inaccessible to students and unauthenticated users via both UI and direct API calls.
- Attempt privilege escalation by modifying client data (e.g., trying to assign self as admin in requests or JS console) and verify the server denies.
- Test concurrent session limits: log in as the same user in multiple browsers/devices, and verify new sessions invalidate the oldest sessions as per config.
- Attempt SQL injection (e.g., using ' OR 1=1 --) in form/search fields and all API endpoints and confirm no SQL errors or data leaks occur.

- Input HTML/JavaScript payloads into all form fields and user-generated content; verify the UI escapes output and no scripts are executed (XSS).
- Post malicious content (e.g., `<script>alert('xss')</script>`) and verify users viewing that content never have JavaScript executed (Stored XSS).
- Upload files containing executable scripts, double extensions, or irregular MIME types; verify uploads are validated, rejected, and never executable by users (malicious file upload).
- Perform valid and invalid payment attempts (using Stripe test cards and error scenarios), checking for transaction correctness, idempotency, and security of data in transit and at rest.
- Trigger high user load (simulated 10 000 concurrent users) and check for bottlenecks in login, dashboard loads, search, and file upload/download.
- Simulate failure/unavailability from Google Calendar API and Stripe API; verify app degrades gracefully, with helpful user messaging and no crashes.

Sekcia s vysokou prioritou vygenerovaného checklistu pre TaskFlow.

Z checklistu je zrejmé, že rizikové pravidlá sa aktivovali konzistentne s tým, čo používateľ v konverzácii uviedol. Pravidlo `sql_injection` sa aktivovalo pre kombináciu `data.has_forms` a `data.has_database`, oba príznaky boli v schéme nastavené na `true`. Podobne sa pravidlo `stored_xss` aktivovalo pre kombináciu používateľsky generovaného obsahu a HTML výstupu. Pravidlá pre integrácie s tretími stranami priniesli položky o správaní pri výpadku Google Calendar a Stripe, čo zodpovedá menám služieb, ktoré používateľ uviedol pri otázke o integráciách.

Položka o súbežných používateľoch obsahuje konkrétne číslo 10 000, ktoré používateľ uviedol pri otázke o výkonnostných požiadavkách. Tento detail naznačuje, že generátor checklistu nepracuje len s booleanovskými hodnotami, ale do textu položiek vkladá aj konkrétne hodnoty zo schémy. To z pohľadu používateľa znamená, že checklist nie je generický, ale priamo odkazuje na to, čo o projekte povedal.

6.2.2 CLI nástroj cez OtherAgent

Druhým príkladom je projekt CLITool, ktorý slúži na overenie dynamického generovania špecialistu. CLI nástroj nespadá do žiadnej zo štyroch preddefinovaných kategórií,

takže RouterService ho zaradil do kategórie `OTHER` a riadenie prevzal OtherAgent. Pre tohto agenta neexistuje predpripravená schéma, preto SpecialistGenerator vyrobil novú schému počas behu na základe používateľského opisu projektu.

CLITool je hypotetický príkazový nástroj na správu nasadení aplikácií vo viacerých cloudových prostrediach. Cieľovými používateľmi sú DevOps inžinieri a backend vývojári, ktorí ho zapájajú do CI/CD pipeline. Nástroj je napísaný v jazyku Go a distribuovaný ako tarball (.tar.gz). Aktuálna verzia je 2.4.1 a dokumentácia je hostovaná na <https://docs.clitool.io>. Podporuje autentifikáciu, viaceré používateľské role, čítanie a zápis konfiguračných súborov vo viacerých formátoch, plugin systém, lokalizáciu a integrácie s CI/CD platformami.

Priebeh konverzácie

Konverzácia začala odoslaním úvodnej správy, na ktorú reagoval OtherAgent svojou prvou otázkou. Skrátený úvodný úsek je v ukážke 6.2.

Používateľ: I'm building a CLI Tool

Chatbot: What's the name of the CLI tool, and what does it do?

Používateľ: Name is CLITool, it's a command-line tool for managing application deployments across multiple cloud environments. Intended users are DevOps engineers and backend developers automating CI/CD pipelines. Primary language is Go, packaging format is .tar, current version is 2.4.1 and documentation url is <https://docs.clitool.io>

Chatbot: Does CLITool require authentication to use, and if so, are there different user roles or authentication methods to support?

Používateľ: Yes, authentication is required. Users must log in before running any deployment commands. The tool supports two authentication methods: API token (for CI/CD pipelines and automated scripts) and OAuth2 via SSO. Multiple user roles exist and there are three roles: viewer (read-only access to deployment status), deployer (can trigger deployments and roll-backs), and admin (full access including environment configuration and user management). Interactive sessions expire after 8 hours of inactivity. The CLI does not auto-lock mid-session, but a new login is required once the session expires.

Obr. 6.2: Skrátený úsek konverzácie pre projekt CLITool.

Konverzácia ďalej pokrývala podporované formáty konfiguračných súborov, prácu s veľkými dátami, plugin systém, podporu lokalizácie do štyroch jazykov, integrácie

s CI/CD platformami (GitHub Actions, GitLab CI, CircleCI), spôsoby distribúcie a inštalácie (Homebrew, apt, rpm, Scoop, Docker), podporované architektúry (x86_64, ARM64, ARMv7) a požiadavky na výkon a stabilitu pri dlhotrvajúcich operáciách.

Pri overovaní boli zaznamenané dve pozorovania. Po prvé, chatbot niekedy kládol otázky, ktoré boli pomerne široké a pokrývali viacero polí schémy naraz, ako je vidieť v druhej otázke vyššie. Používateľ na ne reagoval rozsiahlou odpoveďou a ValidationService z nej extrahoval jednotlivé hodnoty s rôznymi skóre dôvery. Príklad rozkladu druhej odpovede používateľa na polia bloku `auth` je v tabuľke 6.1.

Tabuľka 6.1: Rozklad odpovede používateľa o autentifikácii na jednotlivé polia schémy.

Pole schémy	Extrahovaná hodnota
<code>auth.has_auth</code>	true
<code>auth.auth_types</code>	API token, OAuth2 (SSO)
<code>auth.has_roles</code>	true
<code>auth.roles</code>	viewer, deployer, admin
<code>auth.session_management.has_session_expiry</code>	true (8 hours)
<code>auth.session_management.has_auto_lock</code>	false

Po druhé, v niektorých prípadoch sa chatbot pýtal na informácie, ktoré používateľ už uviedol v úvodnej odpovedi. Napríklad pole `context.documentation_url` bolo súčasťou úvodnej odpovede, ale otázka *What should the documentation URL be for CLITool?* sa objavila neskôr v konverzácii. Podobne sa zopakovala otázka o tom, kto nástroj používa, hoci v úvodnej odpovedi bolo uvedené *Intended users are DevOps engineers and backend developers*. Príčinou bolo, že ValidationService pri prvej iterácii nezachytil všetky polia z rozsiahlej úvodnej odpovede. Tieto polia tak zostali medzi nevyplnenými a chatbot sa na ne neskôr opýtal opätovne. Toto pozorovanie je rozobraté v sekcii 6.6 ako jedno z identifikovaných obmedzení.

Vygenerovaná schéma a doménové pravidlá

Pri overovaní som sledoval, či SchemaGenerator vytvorí schému, ktorá je svojou štruktúrou podobná predpripraveným schémam ostatných špecialistov. Vygenerovaná schéma obsahovala blok `context` s poľami pre názov nástroja, opis, cieľových používateľov, programovací jazyk, formát balíčka, verziu a URL dokumentácie. Ďalšie bloky sa týkali autentifikácie, práce so súbormi, integrácií s CI/CD platformami, plugin systému, lokalizácie a výkonu. Štruktúra zodpovedala vzoru zabudovaných špecialistov, prepínače `has_*` fungovali tak, ako bolo navrhnuté, a podmienené bloky sa vynechávali správne.

HintsGenerator pripravil pre každé pole krátky popis, ktorý ConversationalAgent neskôr použil pri formulovaní otázok. Napríklad pole `environment.has_localization` dostalo nápovedu *whether localization is supported by the CLI tool* a pole `context.packaging_format` nápovedu *format in which the CLI tool is packaged*. DomainRulesGenerator vytvoril pokyn, ktorý ConversationalAgentu signalizoval, že pri CLI nástroji má prednostne klásť otázky o spôsoboch inštalácie, podporovaných platformách, integráciách s CI/CD, autocompletion v shelloch a o správaní v skriptovom režime. Funkčnosť týchto pokynov potvrdzujú aj otázky, ktoré chatbot v konverzácii skutočne kládol: pýtal sa na verbose mode, autocompletion v bash/zsh/fish, lokalizáciu CLI správ, balíčkovacie systémy a podporované architektúry, ktoré sa pri webovej alebo mobilnej aplikácii nevyskytujú.

Vygenerovaný checklist

Po vyplnení schémy systém zostavil checklist, ktorého sekcia s vysokou prioritou je nižšie.

[HIGH PRIORITY]

- Verify that CLI Tool correctly enforces API token and OAuth2 (SSO) authentication on all operational commands (e.g., rollout, rollback, health check).
- Confirm all role-based access controls for viewer, deployer, and admin are enforced on respective commands (e.g., only admins can change deployment configurations).
- Ensure session expiry correctly logs out users and invalidates tokens across all commands with both authentication types.
- Test interactive mode: verify that prompts, confirmations, and abort flows work, including edge cases (unexpected input, empty/invalid responses).
- Verify config file support for supported formats (YAML, JSON, TOML, plain text): tool can read/write each as configuration, errors are informative for malformed files.
- Check command autocompletion works for top-level and sub-commands, flags, and file paths on bash, zsh, and Windows shell.
- Confirm correct operation, including rollback and health check, with very large deployment files (on disk and via stdin).

- Validate import/export and cloud sync of deployment configurations, including error handling for upload/download failures and sync conflicts.
- Confirm that only supported file operations (import, export, edit) are allowed for YAML, JSON, TOML, and plain text files.
- Check accurate and secure log output in both verbose and non-verbose modes, ensuring no sensitive data (tokens/secrets) are logged.
- Verify crash reporting sends relevant error data, truncates PII, and prompts user appropriately per documentation.
- Test installation and basic operation of CLI Tool on Linux, macOS, and Windows, including via .tar.gz and all listed package managers.
- Validate Docker, Homebrew, apt, rpm, and Scoop installation images/packages for correct files, permissions, and path configurations.
- Check that CLI Tool launches and responds on all supported architectures (x86_64, ARM64, ARMv7) and OSes.
- Confirm primary commands function identically with all supported localization languages (English, German, French, Japanese).

Sekcia s vysokou prioritou vygenerovaného checklistu pre CLITool.

Checklist obsahuje položky špecifické pre CLI nástroj, ktoré sa v checklistoch pre webovú alebo mobilnú aplikáciu nevyskytujú. Patria sem autocompletion v bash, zsh a Windows shell, overenie inštalácie cez Docker, Homebrew, apt, rpm a Scoop, kontrolu správania na rôznych procesorových architektúrach a overenie funkcie príkazov v lokalizovaných verziách. Položky sa zároveň odvolávajú na konkrétne hodnoty zo schémy. V testoch autentifikácie sú vymenované všetky tri role, v testoch lokalizácie všetky štyri jazyky a v testoch architektúr všetky tri uvedené platformy. Tento prístup zodpovedá pozorovaniu z prvého príkladu (sekcia 6.2.1): generátor checklistu nepracuje len s booleanovskými hodnotami, ale do textu vkladá aj konkrétne údaje, ktoré používateľ uviedol.

6.3 Overenie s externým používateľom

Okrem demonštračných scenárov pripravených autorom práce bol systém overený aj jedným externým používateľom. Cieľom tohto overenia bolo získať pozorovania, ktoré by sa pri testovaní samotným autorom prejavili menej zreteľné, predovšetkým zrozumiteľnosť otázok chatbota, prirodzenosť konverzačného toku a vnímaná použiteľnosť.

6.3.1 Profil testera a priebeh sedenia

Testerom bol študent tretieho ročníka informatiky so skúsenosťami s testovaním softvéru na úrovni školských cvičení a s každodenným používaním AI chatbotov. Pred sedením nemal o internej architektúre systému žiadne informácie a o testovanom riešení sa dozvedel až pri samotnom sedení.

Tester opisoval hypotetický backendový systém pre e-shop, napísaný v jazyku Python. Aplikácia mala podobu API a poskytovala CRUD operácie nad používateľmi a produktmi, funkcionality nákupného košíka a proces checkoutu s vytvorením objednávky. RouterService zaradil tento projekt do kategórie API a riadenie prevzal APIAgent.

Sedenie pozostávalo z jedinej konverzácie, počas ktorej tester samostatne odpovedal na otázky chatbota. Po jej ukončení vyplnil dotazník zameraný na štyri oblasti: používateľské rozhranie, komunikáciu s chatbotom, technickú stránku riešenia a kvalitu vygenerovaného checklistu.

6.3.2 Pozorovania k používateľskému rozhraniu

Tester označil prihlasovací proces aj vytvorenie novej konverzácie za úplne zrozumiteľné. Tlačidlá pre rýchly štart pochopil bez vysvetlenia, čo sa zhoduje so zámerom, s akým boli do rozhrania zaradené.

Z pohľadu úprav predložil tester konkrétny návrh: pri každej otázke chatbota by sa nad textovým poľom mali zobrazovať štyri tlačidlá, z toho tri s navrhovanými odpoveďami a jedno na zadanie vlastnej odpovede. Návrh vychádza z pozorovania, že niektoré otázky chatbota sú formulované široko a tester nemal istotu, akú úroveň detailu odpoveď vyžaduje. Predpripravené možnosti by tento problém znížili a zároveň by skrátili dĺžku konverzácie. Návrh predstavuje zmysluplné rozšírenie pre ďalšie iterácie systému, hoci si vyžaduje, aby ConversationalAgent okrem samotnej otázky generoval aj zoznam pravdepodobných odpovedí, čo znamená dvojnásobnú prácu pre jazykový model pri každej iterácii.

6.3.3 Pozorovania ku komunikácii s chatbotom

Tester označil približne 90% otázok za zrozumiteľné. Konkrétnu otázku, ktorú vnímal ako nejasnú, si po sedení nepamätal, takže nebolo možné identifikovať jej presnú formuláciu ani ju lokalizovať v schéme.

Tempo konverzácie hodnotil tester ako primerané. Chatbot kládol naraz najviac dve otázky, čo je v súlade s obmedzením definovaným v prompte ConversationalAgent (sekcia 5). Celkovú dĺžku konverzácie však vnímal ako dlhšiu, než pôvodne predpokladal. Pred sedením očakával dĺžku v rádoch jednotiek minút, reálna dĺžka sa však

pohybovala v rozsahu pätnástich minút.

Najvýraznejším pozorovaním v oblasti komunikácie bola opakovaná otázka chatbota týkajúca sa predpokladaného počtu požiadaviek za minútu (rate limiting). Tester tento údaj uviedol už v skoršej fáze konverzácie, no chatbot si ho vyžiadal celkovo trikrát. Toto pozorovanie sa zhoduje s javom popísaným v sekcii 6.5 a potvrdzuje, že problém s neúplným zachytením v poli `ValidationService` nie je obmedzený na sedenie autora. Prejavuje sa aj pri externom používateľovi.

6.3.4 Pozorovania k rýchlosti odozvy

Rýchlosť odpovedí chatbota tester ohodnotil hodnotou 3,5 z 5. V rozhovore po sedení spresnil, že odozvy vnímal ako pomalšie, no systém zostával použiteľný. Konkrétny rozdiel medzi očakávaním a realitou bol značný. Pri jednoduchších otázkach predpokladal odozvu v rádoch jednotiek sekúnd, reálne odpovede prichádzali približne po pätnástich sekundách.

Príčinou je primárne použitie lokálneho jazykového modelu spúšťaného prostredníctvom Ollama na hardvéri vývojového počítača. Pri nasadení s rýchlejšim modelom alebo na výkonnejšej infraštruktúre by sa rozdiel mal zmierniť. Toto pozorovanie je rozšírením obmedzenia popísaného v sekcii 6.6 (Závislosť kvality od jazykového modelu), pretože ukazuje, že voľba modelu ovplyvňuje nielen kvalitu výstupu, ale aj subjektívne vnímanú použiteľnosť systému.

Žiadna technická chyba ani zlyhanie aplikácie sa počas sedenia nevyskytli.

6.3.5 Pozorovania k vygenerovanému checklistu

Vygenerovaný checklist tester ohodnotil ako relevantný pre svoj projekt (4 z 5). Žiadne položky neoznačil ako irelevantné. Neuvedol prípad, kedy by sa do checklistu dostala oblasť testovania, ktorá sa jeho aplikácie netýka. Prioritizácia HIGH/MEDIUM/LOW podľa testera zodpovedala jeho intuitívnemu vnímaniu rizikovosti. Testy v sekcii HIGH považoval za skutočne najpodstatnejšie pre bezpečnosť a stabilitu navrhovaného systému.

Hodnotenie čiastočne potvrdzuje predpoklad z navrhovanej fázy, podľa ktorého má kombinácia podmienených blokov a rizikových pravidiel (sekcia 5) zabezpečiť, že do výstupu sa dostanú len testovacie scenáre relevantné pre konkrétny projekt. Pri jednom externom používateľovi tento mechanizmus splnil zámer.

6.3.6 Súhrn pozorovaní z externého overenia

Overenie s externým používateľom potvrdilo dve pozorovania zaznamenané už pri testovaní autorom. Opakované otázky na informácie, ktoré používateľ uviedol skôr, a pri-

merané tempo konverzácie z hľadiska počtu otázok za jednu iteráciu. Zároveň prinieslo dve nové pozorovania, ktoré pri samotestovaní autorom neboli zreteľné: subjektívne vnímanie pomalosti odozvy ako limitujúceho faktora a konkrétny návrh na rozšírenie rozhrania o predpripravené možnosti odpovedí.

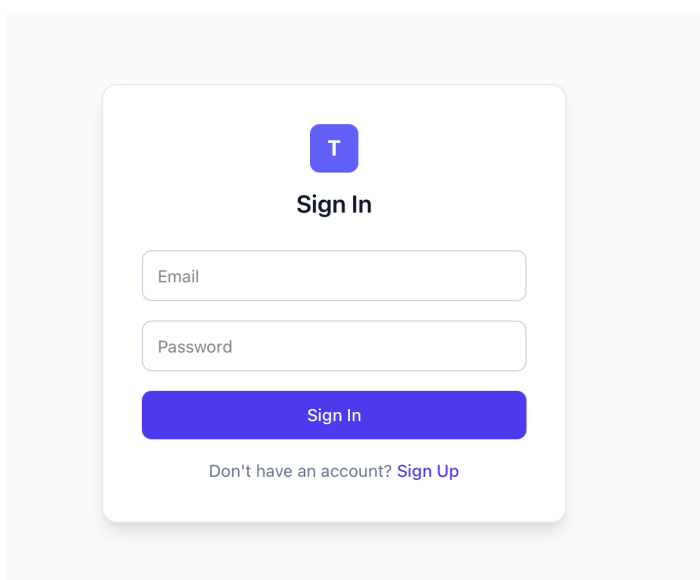
Vzhľadom na to, že overenie prebehlo s jedným externým používateľom, výsledky majú demonštračný, nie štatisticky reprezentatívny charakter. Pre robustnejšie zhodnotenie použiteľnosti by bolo potrebné rozsiahlejšie používateľské testovanie.

6.4 Screenshoty rozhrania

V tejto sekcii uvádzam ukážky z používateľského rozhrania aplikácie. Vybrané boli štyri screenshoty pokrývajúce celý cyklus práce so systémom: prihlásenie, prvé otvorenie aplikácie, stav po vygenerovaní checklistu a podobu exportovaného XLSX súboru.

6.4.1 Prihlasovacie rozhranie

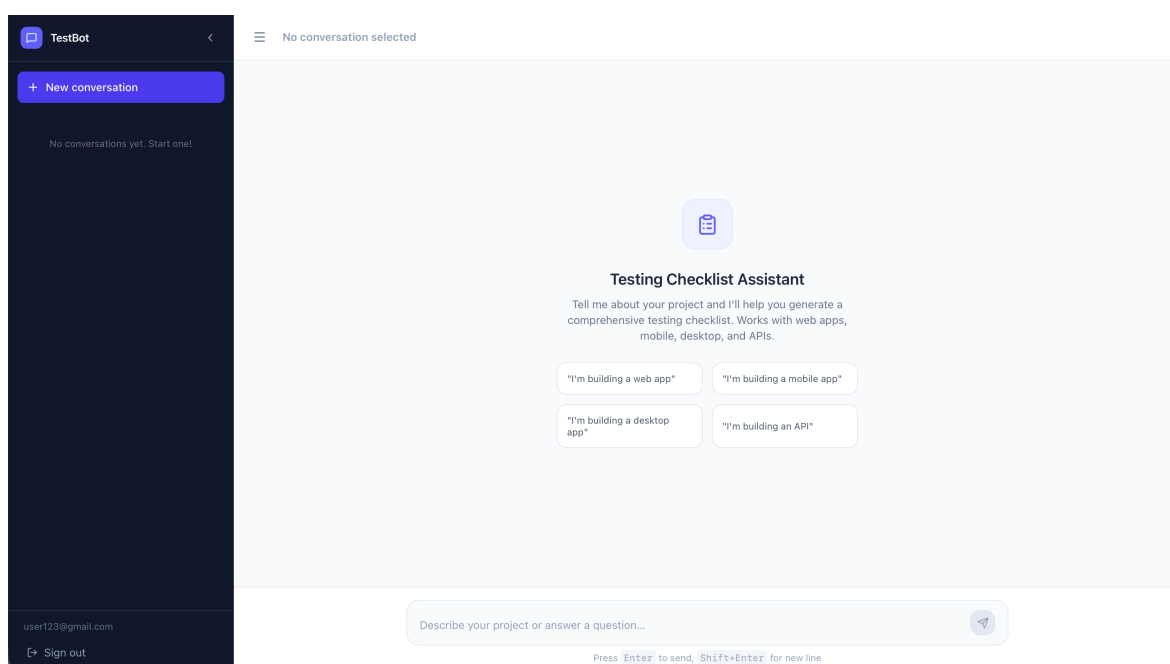
Pri prvom otvorení aplikácie sa používateľovi zobrazí prihlasovací formulár (obrázok 6.3). Formulár obsahuje pole pre email, pole pre heslo a odkaz na registráciu pre nových používateľov. Autentifikáciu zabezpečuje služba Supabase Auth. Bez prihlásenia sa do hlavného rozhrania aplikácie nedá dostať, čo zabezpečuje, že každá konverzácia patrí konkrétnemu používateľovi a je dostupná výhradne jemu.

The image shows a 'Sign In' form centered on a light gray background. At the top of the form is a blue square icon with a white letter 'T'. Below the icon is the text 'Sign In' in bold. There are two input fields: 'Email' and 'Password', both with light gray borders. Below the 'Password' field is a blue button with the text 'Sign In' in white. At the bottom of the form, there is a link that says 'Don't have an account? Sign Up'.

Obr. 6.3: Prihlasovací formulár.

6.4.2 Hlavné okno po prihlásení

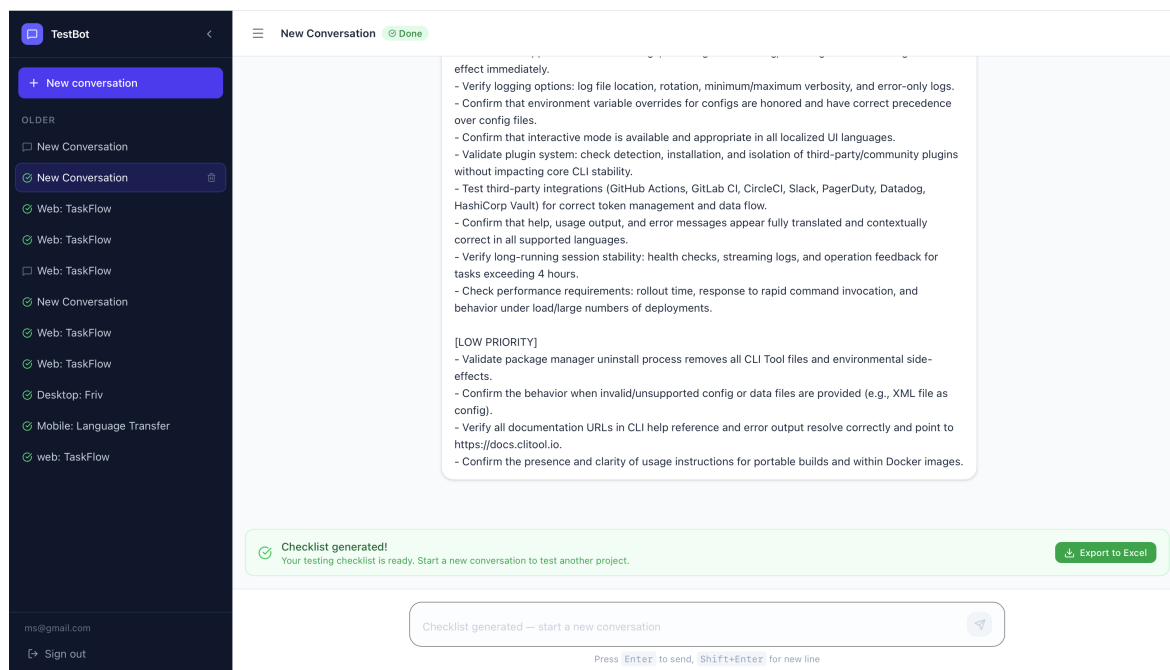
Po úspešnom prihlásení sa otvorí hlavné okno aplikácie (obrázok 6.4). Používateľské rozhranie je rozdelené na dve časti. Bočný panel vľavo obsahuje tlačidlo na vytvorenie novej konverzácie a zoznam predchádzajúcich konverzácií, ktorý je v tomto prípade prázdny. V dolnej časti panela je emailová adresa prihláseného používateľa a tlačidlo na odhlásenie. Hlavná časť obrazovky zobrazuje uvítaciu obrazovku s krátkym opisom funkcionality a štyrmi tlačidlami na rýchly štart konverzácie. Tieto tlačidlá zodpovedajú štyrom preddefinovaným typom aplikácií, ktoré chatbot pozná. Pod uvítacou obrazovkou je textové pole na napísanie vlastnej správy, ktoré používateľ môže využiť, ak jeho projekt do žiadnej z týchto kategórií nezapadá.



Obr. 6.4: Hlavné okno aplikácie po prihlásení a vytvorení nového účtu, ešte bez konverzácií.

6.4.3 Zoznam konverzácií a stav po dokončení

Obrázok 6.5 zachytáva stav aplikácie po dlhšom používaní. V bočnom paneli sú viaceré predchádzajúce konverzácie, pričom dokončené sú odlíšené zelenou ikonou a nedokončené sivou. V hlavnom okne je vidieť koniec konverzácie pre projekt CLITool s vygenerovaným checklistom v sekcii LOW PRIORITY a v dolnej časti zelený banner s oznámením, že checklist bol vygenerovaný. Banner obsahuje aj tlačidlo na export do XLSX súboru. Textové pole na zadávanie nových správ je v tomto stave zablokované, čo používateľovi signalizuje, že konverzácia bola uzavretá. Ak chce používateľ pokračovať s iným projektom, musí založiť novú konverzáciu.



Obr. 6.5: Stav aplikácie po dokončení konverzácie pre projekt CLITool.

6.4.4 Exportovaný checklist

Po stlačení tlačidla *Export to Excel* sa stiahne XLSX súbor obsahujúci hotový checklist (obrázok 6.6). Súbor má dva stĺpce. Prvý uvádza prioritu testovacieho prípadu (HIGH, MEDIUM, LOW), druhý obsahuje samotný popis testu. Riadky sú farebne odlíšené podľa priority: vysoká je označená červenou, stredná žltou a nízka zelenou. Toto vizuálne odlíšenie umožňuje používateľovi rýchlo nájsť testy, na ktoré sa má sústrediť ako prvé. Tabuľka zachováva poradie položiek z textu konverzácie, takže používateľ môže výstup priamo skopírovať do testovacieho nástroja, prípadne si ho doplniť o vlastné poznámky.

	A	B
1	Priority	Test Case
2	HIGH	Verify that CLI Tool correctly enforces API token and OAuth2 (SSO) authentication on all operational commands (e.g., rollout, rollback, health check).
3	HIGH	Confirm all role-based access controls for viewer, deployer, and admin are enforced on respective commands (e.g., only admins can change deployment configurations).
4	HIGH	Ensure session expiry correctly logs out users and invalidates tokens across all commands with both authentication types.
5	HIGH	Test interactive mode: verify that prompts, confirmations, and abort flows work, including edge cases (unexpected input, empty/invalid responses).
6	HIGH	Verify config file support for supported formats (YAML, JSON, TOML, plain text): tool can read/write each as configuration, errors are informative for malformed files.
7	HIGH	Check command autocompletion works for top-level and sub-commands, flags, and file paths on bash, zsh, and Windows shell.
8	HIGH	Confirm correct operation, including rollback and health check, with very large deployment files (on disk and via stdin).
9	HIGH	Validate import/export and cloud sync of deployment configurations, including error handling for upload/download failures and sync conflicts.
10	HIGH	Confirm that only supported file operations (import, export, edit) are allowed for YAML, JSON, TOML, and plain text files.
11	HIGH	Check accurate and secure log output in both verbose and non-verbose modes, ensuring no sensitive data (tokens/secrets) are logged.
12	HIGH	Verify crash reporting sends relevant error data, truncates PII, and prompts user appropriately per documentation.
13	HIGH	Test installation and basic operation of CLI Tool on Linux, macOS, and Windows, including via .tar.gz and all listed package managers.
14	HIGH	Validate Docker, Homebrew, apt, rpm, and Scoop installation images/packages for correct files, permissions, and path configurations.
15	HIGH	Check that CLI Tool launches and responds on all supported architectures (x86_64, ARM64, ARMv7) and OSes.
16	HIGH	Confirm primary commands function identically with all supported localization languages (English, German, French, Japanese).
17	MEDIUM	Validate shell integration: output formatting, colored output, and command piping work as documented on each platform.
18	MEDIUM	Test correct application of feature flags, ensuring that enabling/disabling features via flags takes effect immediately.
19	MEDIUM	Verify logging options: log file location, rotation, minimum/maximum verbosity, and error-only logs.
20	MEDIUM	Confirm that environment variable overrides for configs are honored and have correct precedence over config files.
21	MEDIUM	Confirm that interactive mode is available and appropriate in all localized UI languages.
22	MEDIUM	Validate plugin system: check detection, installation, and isolation of third-party/community plugins without impacting core CLI stability.
23	MEDIUM	Test third-party integrations (GitHub Actions, GitLab CI, CircleCI, Slack, PagerDuty, Datadog, HashiCorp Vault) for correct token management and data flow.
24	MEDIUM	Confirm that help, usage output, and error messages appear fully translated and contextually correct in all supported languages.
25	MEDIUM	Verify long-running session stability: health checks, streaming logs, and operation feedback for tasks exceeding 4 hours.
26	MEDIUM	Check performance requirements: rollout time, response to rapid command invocation, and behavior under load/large numbers of deployments.
27	LOW	Validate package manager uninstall process removes all CLI Tool files and environmental side-effects.
28	LOW	Confirm the behavior when invalid/unsupported config or data files are provided (e.g., XML file as config).
29	LOW	Verify all documentation URLs in CLI help reference and error output resolve correctly and point to https://docs.clitool.io .
30	LOW	Confirm the presence and clarity of usage instructions for portable builds and within Docker images.

Obr. 6.6: Exportovaný checklist v podobe XLSX súboru.

6.5 Pozorovania k správaniu systému

V tejto sekcii sú zhrnuté pozorovania zaznamenané počas overovania. Niektoré sa týkajú vlastností, ktoré fungovali tak, ako boli navrhnuté, iné odhalili limity, ktoré bližšie rozoberám v sekcii 6.6.

Klasifikácia typu aplikácie

RouterService spoľahlivo zaraďoval projekt do správnej kategórie vtedy, keď používateľ v úvodnej správe pomenoval, o aký typ aplikácie ide. Pri opisoch ako *I'm building a CLI tool for managing deployments* alebo *e-shop with payments* sa klasifikácia zhodovala s očakávanou kategóriou. Slabšie výsledky boli pri vágnych alebo všeobecne formulovaných opisoch, kde používateľ nepoužil žiaden charakteristický termín. V takých prípadoch RouterService priradľoval kategóriu OTHER a OtherAgent musel doménu odhadnúť prostredníctvom SpecialistGenerator. Ide síce o správanie, na ktoré bol systém navrhnutý, no výsledná schéma nebola taká presná ako pri zabudovaných špecialistoch, pretože SchemaGenerator pracoval s menším množstvom doménových informácií.

Z toho pozorovania vyplýva, že efektivita konverzácie závisí od kvality úvodnej správy. Pri konkrétnom opise projektu systém zachytí typ správne a pripraví preňho relevantnú schému. Pri všeobecnom opise sa zaraďí do dynamickej vetvy a kvalita výstupu klesá.

Podmienené bloky

Prepínače `has_*` v schéme fungovali pri všetkých testovaných projektoch tak, ako boli navrhnuté. Ak používateľ uviedol, že aplikácia nemá určitú vlastnosť (napríklad autentifikáciu alebo platby), celý príslušný blok schémy sa pri ďalších otázkach preskočil a vo výslednom checkliste sa neobjavili položky, ktoré by sa k tejto vlastnosti viazali. Toto pozorovanie sa zhodovalo s návrhom v `SessionManager`, ktorý vetvenie schémy rieši na základe hodnoty prepínača. Žiaden z testovaných projektov neodhalil prípad, kde by sa irelevantná otázka dostala do konverzácie alebo kde by sa irelevantné riziko aktivovalo v checkliste.

Opakujúce sa otázky

Pri všetkých testovaných projektoch sa vyskytol prípad, kde sa chatbot opýtal na informáciu, ktorú používateľ uviedol už skôr v konverzácii. Najčastejšie sa to týkalo polí, ktoré používateľ vložil do úvodnej rozsiahlej odpovede a `ValidationService` ich pri prvej iterácii nezachytil. Príklad takéhoto správania je uvedený pri projekte `CLITool` v sekcii 6.2.2. Podobné prípady som zaznamenal aj pri ostatných projektoch.

Príčinou je samotný validačný proces. `ValidationService` spracúva odpoveď používateľa cez jazykový model a do schémy zapíše len tie polia, pri ktorých má dostatočne vysokú mieru istoty. Ak je odpoveď príliš rozsiahla alebo používateľ informáciu spomenie len okrajovo, model môže pole prehliadnúť alebo mu priradiť nízku istotu. Pole tak ostane nevyplnené a chatbot sa naň o niekoľko krokov neskôr opýta opätovne. Toto správanie nemá vplyv na výsledný checklist, pretože pole sa nakoniec vyplní, ale negatívne ovplyvňuje plynulosť konverzácie.

Generovaný checklist a export

Vygenerované checklisty obsahovali pri každom testovanom projekte konkrétne hodnoty zo schémy, nielen všeobecné formulácie. Mená integrácií, čísla, role a jazyky, ktoré používateľ uviedol, sa objavili priamo v texte testovacích položiek. Toto pozorovanie

sa potvrdilo pri projekte TaskFlow (sekcia 6.2.1) aj pri CLITool (sekcia 6.2.2).

Export do XLSX súboru fungoval pri všetkých testovaných projektoch bez problémov. Parser založený na regulárnych výrazoch správne rozdelil text checklistu na dvojice (priorita, testovací prípad) a všetky položky z troch prioritných sekcií sa preniesli do hárka s farebným odlíšením podľa priority. Pri žiadnom z testovaných projektov sa neukázal prípad, keď by sa časť položiek pri exporte stratila alebo bola nesprávne zaradená.

6.6 Identifikované obmedzenia

V tejto sekcii zhrniem obmedzenia systému, ktoré boli zaznamenané počas overovania alebo ktoré vyplývajú zo samotného návrhu. Niektoré z nich možno riešiť úpravou implementácie, iné súvisia so samotnou povahou riešenia, ktoré stojí na jazykovom modeli a na vlastnom úsudku používateľa.

6.6.1 Závislosť výstupu od formulácie úvodnej správy

Kvalita celej konverzácie závisí od toho, ako konkrétne používateľ už v prvej správe pomenuje, o aký projekt ide. Pri jasných opisoch RouterService projekt zaradí do správnej kategórie a aktivuje sa špecialista s pripravenou schémou. Pri vágnych opisoch sa projekt zaradí do kategórie `OTHER` a SchemaGenerator musí schému poskladať z menšieho množstva informácií. Výsledná schéma je v takom prípade menej presná a checklist môže obsahovať položky, ktoré sa daného projektu netýkajú, alebo naopak môže vynechať relevantné riziká. Toto obmedzenie možno čiastočne zmierniť tým, že úvodné rozhranie aplikácie predkladá používateľovi štyri tlačidlá s preddefinovanými kategóriami, ale neeliminuje sa, pretože za týmito tlačidlami nasleduje rovnaký textový vstup.

6.6.2 Neúplné zachytenie polí z rozsiahlych odpovedí

ValidationService nie vždy zachytí všetky polia z dlhej odpovede používateľa. Ak používateľ v jednej správe odpovie naraz na štyri alebo päť otázok, niektoré polia môžu zostať nevyplnené a chatbot sa na ne neskôr opýta opätovne. Tento jav som pozoroval pri všetkých testovaných projektoch a podrobnejšie je rozoberaný v sekcii 6.5. Konečný checklist tým netrpí, pretože polia sa nakoniec vyplnia, no konverzácia pôsobí menej plynulo, najmä vtedy, keď sa chatbot opýta na informáciu, ktorú používateľ považoval za už vybavenú.

6.6.3 Závislosť kvality od jazykového modelu

Systém volá lokálny jazykový model prostredníctvom Ollama vo viacerých komponentoch, najmä v službe RouterService, ConversationalAgent, ValidationService a v generátoroch dynamického špecialistu. Kvalita výstupu týchto komponentov priamo závisí od schopností použitého modelu. Pri overovaní bolo vyskúšaných viacero modelov dostupných prostredníctvom Ollama a zaznamenalo sa, že niektoré z nich mali problémy s extrakciou polí v ValidationService, prípadne generovali schémy, ktoré nedodržiavali očakávaný formát. Stabilný výsledok bol dosiahnutý len pri konkrétnej kombinácii modelov, pre ktoré boli ladené aj systémové prompty. Iný používateľ, ktorý systém nasadí s odlišným modelom, môže pozorovať odlišné správanie, a kvalita checklistu sa môže zmeniť bez toho, aby používateľ čokoľvek zmenil v kóde alebo v promptoch.

6.6.4 Statické rizikové pravidlá pri preddefinovaných kategóriách

Pre štyri zabudované kategórie (web, mobile, desktop, API) sú rizikové pravidlá zapísané priamo v kóde. Každé pravidlo má pevne danú podmienku, ktorá určuje, kedy sa má aktivovať, a pevne daný text testovacej položky, ktorý sa v takom prípade pridá do checklistu. Tento prístup zjednodušuje vývoj a zabezpečuje predvídateľnosť výstupu, zároveň však znamená, že pri vzniku nového typu rizika alebo pri zmene odporúčaných postupov je potrebné pravidlá manuálne dopisovať a nasadzovať novú verziu aplikácie. Dynamickejšie správanie poskytuje DomainRulesGenerator pri kategórii OTHER, ten však pracuje na úrovni inštrukcie pre ConversationalAgent, nie na úrovni explicitných pravidiel s presnou podmienkou. Cenou za jeho flexibilitu je teda nižšia presnosť výstupu oproti pevne zapísaným pravidlám.

Záver

Cieľom tejto práce bolo vytvoriť konverzačný systém, ktorý z textového opisu projektu vytvorí testovací checklist, ktorý následne možno použiť ako podklad pri plánovaní testovania.

V úvode práce som sa venoval teoretickým východiskám z oblasti testovania softvéru. Zameral som sa na jeho úlohu v rámci vývojového procesu a dopad kvality testovania na náklady projektu. Opísal som model kvality podľa normy ISO/IEC 25010. Záver úvodu sa zameriava na prehľad vybraných AI nástrojov, ktoré sa využívajú pri vývoji softvéru, konkrétne ChatGPT, GitHub Copilot a Slackbot AI.

V analytickej časti práce som identifikoval problém, s ktorým sa môžu stretnúť najmä malé vývojové tímy a menej skúsení tester. Ide o absenciu nástroja, ktorý by im pomohol pri tvorbe testovacieho plánu. Následne som definoval funkčné a nefunkčné požiadavky pre navrhovaný systém. Riešenie má komunikovať s používateľom pomocou prirodzenej konverzácie, udržiavať si kontext rozhovoru a z voľného opisu projektu vedieť identifikovať relevantné oblasti na testovanie. Výsledkom systému má byť štrukturovaný checklist.

V implementačnej časti som opisoval fungovanie jednotlivých komponentov systému a ich vzájomnú komunikáciu. Medzi komponenty systému patrí API vrstva, Orchestrator, AgentRegistry, špecialisti, RouterService, ConversationalAgent, ValidationService a generátor dynamického špecialistu. Systém je postavený na lokálnych jazykových modeloch spúšťaných prostredníctvom Ollama a všetky potrebné dáta sa uchovávajú v databáze Supabase.

Na overenie funkčnosti systému som použil dva rôzne typy projektov. Prvým bola webová aplikácia TaskFlow, na ktorej bola testovaná najrozsiahlejšia schéma špecialistu. Druhým bol CLI nástroj CLITool, ktorý slúžil na overenie správnosti dynamického vygenerovania špecialistu. V oboch prípadoch bol checklist vygenerovaný s korektnými hodnotami zo schémy a pokrýval relevantné testovacie oblasti pre daný typ aplikácie. Súčasťou overenia bolo aj testovanie s externým používateľom, ktoré potvrdilo, že systém dokáže používať aj osoba bez internej znalosti systému. Zároveň to prinieslo nové pozorovania, ktoré neboli zreteľné pri samotestovaní autora. Používateľ poznamenal predovšetkým rýchlosť odozvy a návrh rozšírenia rozhrania o predpripravené odpovede. Pri overovaní boli zaznamenané aj obmedzenia týkajúce sa neúplného

zachytenia informácií. Kvalita výstupu je zároveň závislá od formulácie odpovedí a konkrétneho jazykového modelu.

Hlavné prínosy práce

Hlavným technickým prínosom práce je návrh a implementácia hybridnej architektúry špecialistov. Pre typy projektov, ktoré sa často opakujú, systém využíva statického špecialistu s vopred definovanými pravidlami a schémou. V prípade výskytu projektu, ktorý nespadá do žiadnej z pripravených kategórií, systém poskytuje dynamické vygenerovanie špecialistu počas behu. Tým pokryje aj domény, ktoré neboli vopred pripravené.

- **Konverzačný prístup k tvorbe testovacieho checklistu.** Používateľ opíše projekt v prirodzenom jazyku bez toho, aby ovládal testovacie metodiky.
- **Dynamické generovanie špecialistu.** Pre projekty, ktoré nepatria do pripravených kategórií, systém počas behu vygeneruje novú schému, nápovedy a doménové pravidlá.
- **Štruktúrovaný a exportovateľný výstup.** Checklist je rozdelený do troch prioritných úrovní a možno ho exportovať ho do XLSX súboru, ktorý je priamo použiteľný v ďalších nástrojoch.
- **Personalizácia výstupu.** Checklist obsahuje konkrétne hodnoty zo schémy (mená integrácií, používateľské role, podporované jazyky a platformy), takže testovacie položky odkazujú na opisovaný projekt, nie na všeobecný šablónovitý zoznam.

Mojím osobným prínosom bolo získanie praktických skúseností s integráciou jazykových modelov do produkčnej aplikácie a so štruktúrovaním promptov tak, aby výstup zodpovedal konkrétnemu formátu (schéma, JSON, formát checklistu).

Možné smery ďalšieho vývoja

Pre ďalšiu prácu na systéme vidím niekoľko smerov, v ktorých by sa dalo posunúť.

Lokalizácia rozhrania a výstupu.

Aktuálna verzia systému je navrhnutá na používanie v angličtine. Rozšírenie podpory o ďalšie jazyky by neznamenal iba preklad textov v používateľskom rozhraní, ale aj úpravu promptov a overenie, že extrakcia polí funguje rovnako spoľahlivo ako v anglickom jazyku.

Podpora viacerých jazykových modelov.

Systém je momentálne prispôsobený pre kombináciu lokálnych jazykových modelov prostredníctvom Ollama. Ďalším krokom by bolo umožniť jednoduchšiu výmenu používaného modelu, vrátane cloudového API, napríklad OpenAI alebo Anthropic. Súčasťou tohto rozšírenia by bolo aj prispôsobenie promptov pre daného sprostredkovateľa zvoleného modelu.

Rozšírenie statických rizikových pravidiel o externé zdroje.

Pravidlá pre pripravených špecialistov sú zapísané priamo v kóde. Vhodným rozšírením by bolo čerpať ich z externých zdrojov, napríklad z OWASP alebo ASVS. Vďaka tomu by sa predišlo manuálnemu aktualizovaniu pravidiel v kóde a zjednodušilo by to tematické pokrytie systému.

Literatúra

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Kent Beck. *Test-driven development: by example*. Addison-Wesley Professional, 2003.
- [3] Cédric Beust and Hani Suleiman. *Next generation Java testing: TestNG and advanced concepts*. Pearson Education, 2007.
- [4] Pierre Bourque and Richard E Fairley. *SWEBOK: guide to the software engineering body of knowledge*. IEEE Computer Society, 2014.
- [5] Capgemini, Sogeti, and OpenText. World quality report 2024–25: New futures in focus. Technical report, Capgemini, 2024. [Online; cit. 2026-05-03].
- [6] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jack Jiang. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software*, 203:111734, 2023.
- [7] Edsger Wybe Dijkstra. Structured programming. 1970.
- [8] Paul M Duvall, Steve Matyas, and Andrew Glover. *Continuous integration: improving software quality and reducing risk*. Pearson Education, 2007.
- [9] GitHub. Writing tests with github copilot, 2026.
- [10] Paul Hamill. *Unit test frameworks: tools for high-quality software development*. O'Reilly Media, Inc.", 2004.
- [11] Antawan Holmes and Marc Kellogg. Automating functional tests using selenium. In *AGILE 2006 (AGILE'06)*. IEEE, 2006.
- [12] International Software Testing Qualifications Board. *Certified Tester Foundation Level Syllabus*. International Software Testing Qualifications Board, 2024.

- [13] Hasnain Iqbal and Muhammad Babar. An approach for analyzing iso/iec 25010 product quality requirements based on fuzzy logic and likert scale for decision support systems. *International Journal of Advanced Computer Science and Applications*, 7(12), 2016.
- [14] Heather A Johnson. Slack. *Journal of the Medical Library Association: JMLA*, 106(1):148, 2018.
- [15] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and individual differences*, 103:102274, 2023.
- [16] Herb Krasner. The cost of poor software quality in the US: A 2022 report. Technical report, Consortium for Information & Software Quality (CISQ), November 2022.
- [17] Waweru Mwaura. *End-to-End Web Testing with Cypress: Explore techniques for automated frontend web testing with Cypress and JavaScript*. Packt Publishing Ltd, 2021.
- [18] Tom Nadarzynski, Oliver Miles, Aimee Cowie, and Damien Ridge. Acceptability of artificial intelligence (ai)-led chatbot services in healthcare: A mixed-methods study. *Digital health*, 5:2055207619871808, 2019.
- [19] PractiTest and Tea-time with Testers. State of testing report 2024. Technical report, PractiTest, 2024. [Online; cit. 2026-05-03].
- [20] Omer Aftab Qureshi, Stephen Wilkins, and Huda Iqbal. Chatbots: Can they satisfy customers in the banking sector? In *BUID Doctoral Research Conference 2023: Multidisciplinary Studies*, pages 360–376. Springer, 2024.
- [21] Yahya Rafique and Vojislav B Mišić. The effects of test-driven development on external quality and productivity: A meta-analysis. *IEEE Transactions on Software Engineering*, 39(6):835–856, 2012.
- [22] Slack. Meet slackbot: Your personal ai agent for work, 2026.
- [23] Slack. Slackbot: Personal ai agent for work, 2026.
- [24] Ian Sommerville. Software engineering 9th edition. *ISBN-10*, 137035152:18, 2011.

- [25] Joseph Weizenbaum. Eliza—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1):36–45, 1966.
- [26] Dave Westerveld. *API Testing and Development with Postman: API creation, testing, debugging, and management made easy*. Packt Publishing Ltd, 2024.
- [27] Lu Xu, Leslie Sanders, Kay Li, and James CL Chow. Chatbot for health care and oncology applications using artificial intelligence and machine learning: systematic review. *JMIR cancer*, 7(4):e27850, 2021.

Príloha A: Zdrojový kód

Zdrojový kód systému opísaného v tejto práci je verejne dostupný v repozitári:

`https://github.com/MatusSL/Bachelors-Project`

Repozitár obsahuje zdrojový kód backendu, frontendu, definície schém jednotlivých špecialistov a pokyny na lokálne spustenie systému uvedené v súbore `README.md`.

Príloha B: Checklisty

V tejto prílohe sú uvedené úplné podoby vygenerovaných checklistov pre tri projekty, ktoré boli použité pri overovaní funkčnosti systému (kapitola 6). Ide o webovú aplikáciu TaskFlow, backendový systém EshopBackend a CLI nástroj CLITool. V hlavnom texte práce boli z priestorových dôvodov uvedené iba sekcie s vysokou prioritou; v tejto prílohe sú jednotlivé checklisty zobrazené v plnom rozsahu vrátane sekcií so strednou a nízkou prioritou.

B.1 Checklist projektu TaskFlow

	A	B	
2	HIGH	Verify that authentication using email/password enforces correct credentials and properly handles failed login attempts (TaskFlow student/admin roles).	
3	HIGH	Test Google OAuth login flow, ensuring tokens are validated and user account is created/linked appropriately.	
4	HIGH	Check that session tokens expire after the configured timeout, and users are logged out and redirected to the login page.	
5	HIGH	Attempt to access the admin panel as a student and as an unauthenticated user—verify access is denied and no privileged actions are exposed.	
6	HIGH	Confirm that admin-only functionality and API routes are inaccessible to students and unauthenticated users via both UI and direct API calls.	
7	HIGH	Attempt privilege escalation by modifying client data (e.g., trying to assign self as admin in requests or JS console) and verify the server denies.	
8	HIGH	Test concurrent session limits: Log in as the same user in multiple browsers/devices, and verify new sessions invalidate the oldest sessions as per config.	
9	HIGH	Attempt SQL injection (e.g., using ``OR 1=1 --` in form/search fields and all API endpoints) and confirm no SQL errors or data leaks occur.	
10	HIGH	Input HTML/JavaScript payloads into all form fields and user-generated content; verify the UI escapes output and no scripts are executed (XSS).	
11	HIGH	Post malicious content (e.g., ` <script>alert('xss')</script> `) and verify users viewing that content never have JavaScript executed (Stored XSS).	
12	HIGH	Upload files containing executable scripts, double extensions, or irregular MIME types; verify uploads are validated, rejected, and never executable by users (Malicious file upload).	
13	HIGH	Intentionally miss required fields or enter boundary/extreme values for all forms and API endpoints; confirm server/client-side validation errors occur.	
14	HIGH	Perform valid and invalid payment attempts (using Stripe test cards and error scenarios), checking for transaction correctness, idempotency, and security of data in transit and at rest.	
15	HIGH	Confirm API endpoints always require authentication/authorization (e.g., student/student+admin roles) and that unauthenticated or unauthorized access returns 401 or 403.	
16	HIGH	Send malformed or overly long input to all REST API endpoints and verify validation errors or 400-level API responses, never server errors.	
17	HIGH	Trigger high user load (simulated 10,000 concurrent users) and check for bottlenecks in login, dashboard loads, search, and file upload/download.	
18	HIGH	Rapidly trigger database write/delete operations and check that data remains consistent, with no lost or duplicated records (Data integrity).	
19	HIGH	Run repeated searches, paginations, and content updates in parallel sessions/users, and verify database and UI remain in a consistent state.	
20	HIGH	Test search functionality with potential injection payloads and ensure results are sanitized and queries are parameterized.	
21	HIGH	Simulate failure/unavailability from Google Calendar API and Stripe API; verify app degrades gracefully, with helpful user messaging and no crashes.	
22	MEDIUM	Log in with "Remember me" enabled; verify session is persistent across browser restarts and doesn't expose session tokens in URLs, cookies are marked secure, and token is invalidated on logout.	

Obr. 6.7: Checklist projektu TaskFlow — sekcia HIGH PRIORITY

22	MEDIUM	Log in with "Remember me" enabled; verify session is persistent across browser restarts and doesn't expose session tokens in URLs, cookies are marked secure, and token is invalidated on logout.	
23	MEDIUM	Download allowed file types as an authorized and unauthorized user; verify files are only accessible to permitted users and never sensitive admin/student data leaks.	
24	MEDIUM	Submit complex search queries ('--', '<script>', emojis, etc.) and verify that results display correctly, with no security or stability issues.	
25	MEDIUM	Log in via UI using correct credentials; start a session, let it expire, and attempt any authenticated action—confirm session is expired and user is prompted to re-authenticate.	
26	MEDIUM	Temporarily disable Google Calendar or Stripe via feature flags; check that associated functionality is consistently clipped from both UI and all API endpoints for all users and roles.	
27	MEDIUM	Verify email notification delivery (e.g., registration, password reset) with SendGrid, including error/invalidation simulations (spam, blocked, etc.) and correct content in English, Slovak, German.	
28	MEDIUM	Test app in staging/production; verify config variables, integrations, and environment-specific features (flags, payment sandbox/live mode) are set correctly in each environment.	
29	MEDIUM	Activate and deactivate all feature flags, confirming that each flag's on/off state affects UI and functionality as intended across roles and devices, with no inconsistent states.	
30	MEDIUM	Induce an unhandled exception and verify error monitoring/alerting solutions capture details and produce developer/operator notifications.	
31	MEDIUM	Test the UI with screen readers, keyboard navigation, and color contrast checkers on Firefox and Safari (desktop and mobile) to confirm accessibility compliance (WCAG 2.1 AA).	
32	MEDIUM	Simulate third-party email, calendar, or payment service failure or latency and verify user is shown meaningful fallback/notification and can retry or proceed safely.	
33	LOW	Intentionally break submitted pagination params (e.g., negative page numbers, large page sizes, duplicate requests) and verify results are correct with no duplication or missing items.	
34	LOW	Change app localization to Slovak and German on both mobile and tablet, and confirm that all UI elements, forms, errors, and emails are correctly translated/adjusted/buttons and no clipped text appears.	
35	LOW	Test the layout across phones and tablets (Firefox/Safari), especially at breakpoints and with RTL/CJK characters, to verify no overlap/breakage of critical UI pieces.	
36	LOW	Enable all UI animations on various devices, checking that performance remains smooth and no lag, jank, or artifacts appear in key interactions.	
37	LOW	Confirm every public page has valid robots.txt, meta tags, canonical links, and structured data/schema to meet SEO requirements (crawlable, indexable, accurate).	
38	LOW	Open TaskFlow in multiple tabs/windows, trigger key user actions (logging in/out, uploading content), and verify analytics tracking records correct page views, events, user IDs, and does not send duplicate or wrong data.	
39			

Obr. 6.8: Checklist projektu TaskFlow — sekcie MEDIUM a LOW PRIORITY

B.2 Checklist projektu EshopBackend

A		B
1	Priority	Test Case
2	HIGH	Verify that a request to POST /v1/users without an API key header returns HTTP 401 Unauthorized.
3	HIGH	Verify that a request to POST /v1/users with an invalid API key header returns HTTP 401 Unauthorized.
4	HIGH	Verify that a request to DELETE /v1/users/{userid} using an API key that does not belong to the target user returns HTTP 403 Forbidden.
5	HIGH	Verify that a bulk delete request to POST /v1/users/bulk-delete with more than 100 user IDs returns HTTP 400 Bad Request and an error message indicating the limit.
6	HIGH	Verify that a bulk delete request to POST /v1/users/bulk-delete with 50 valid and 10 invalid user IDs returns HTTP 207 Multi-Status with appropriate success/failure status for each ID.
7	HIGH	Verify that uploading a file with a .exe extension to POST /v1/files/upload returns HTTP 415 Unsupported Media Type.
8	HIGH	Verify that uploading a file containing a malicious script to POST /v1/files/upload returns HTTP 400 Bad Request and no file is stored.
9	HIGH	Verify that downloading a file via GET /v1/files/{fileid} without an API key returns HTTP 401 Unauthorized.
10	HIGH	Verify that downloading a file that belongs to another user via GET /v1/files/{fileid} returns HTTP 403 Forbidden.
11	HIGH	Verify that sending a search query to GET /v1/products?search=' OR 1=1 -- returns no results and HTTP 200 OK, and does not expose database errors.
12	HIGH	Verify that creating a product with payload {"name":"Test","price":1; DROP TABLE products;} returns HTTP 400 Bad Request and does not alter the database.
13	HIGH	Verify that creating a product with a negative price returns HTTP 400 Bad Request and an error message indicating invalid price.
14	HIGH	Verify that the /v1/users endpoint does not return any sensitive fields such as password_hash or ssn in its JSON response.
15	HIGH	Verify that any error response from the API follows the consistent error format: {"error":{"code":"ERR_CODE","message":"Error message"}} and does not include stack traces or sensitive data.
16	HIGH	Verify that a GET request to /v1/products?page=1&pageSize=1000 completes within 2 seconds under a load of 4 requests per second.
17	HIGH	Verify that a POST request to /v1/orders/checkout completes within 1 second and returns HTTP 200 OK.
18	HIGH	Verify that creating a user with payload {"username":"admin","is_admin":true} results in the is_admin field being ignored and the user not having admin privileges.
19	HIGH	Verify that updating a user with payload {"username":"john","extra_field":"value"} does not persist extra_field in the database.
20	HIGH	Verify that an OPTIONS preflight request from origin https://malicious.com to any endpoint returns HTTP 403 Forbidden and does not include Access-Control-Allow-Origin header.
21	HIGH	Verify that an OPTIONS preflight request from origin https://trusted.com to any endpoint returns HTTP 200 OK and includes Access-Control-Allow-Origin: https://trusted.com header.
22	MEDIUM	Verify that a GET request to /v1/products?page=1&pageSize=20 and /v1/products?page=2&pageSize=20 return distinct sets of products and that the total count reported matches the number of products in the database.
23	MEDIUM	Verify that a request to an undefined endpoint /v1/undefined returns HTTP 404 Not Found with the JSON error format {"error":{"code":"ERR_NOT_FOUND","message":"Endpoint not found"}}.
24	MEDIUM	Verify that a request to /v1/users with a new API version /v2/users returns HTTP 404 Not Found, confirming that versioning is enforced.
25	MEDIUM	Verify that sending an order creation message to the message queue results in exactly one order record in the database, and that re-sending the same message does not create a duplicate order.
26	MEDIUM	Verify that the staging environment uses the same API key and base URL as production, and that environment-specific configuration files do not expose sensitive data.
27	LOW	Verify that the mock API for POST /v1/products returns the same JSON structure and status code as the real API when creating a product.
28		

Obr. 6.9: Checklist projektu EshopBackend

B.3 Checklist projektu CLITool

	A	B
1	Priority	Test Case
2	HIGH	Verify that CLI Tool correctly enforces API token and OAuth2 (SSO) authentication on all operational commands (e.g., rollout, rollback, health check).
3	HIGH	Confirm all role-based access controls for viewer, deployer, and admin are enforced on respective commands (e.g., only admins can change deployment configurations).
4	HIGH	Ensure session expiry correctly logs out users and invalidates tokens across all commands with both authentication types.
5	HIGH	Test interactive mode: verify that prompts, confirmations, and abort flows work, including edge cases (unexpected input, empty/invalid responses).
6	HIGH	Verify config file support for supported formats (YAML, JSON, TOML, plain text): tool can read/write each as configuration, errors are informative for malformed files.
7	HIGH	Check command autocompletion works for top-level and sub-commands, flags, and file paths on bash, zsh, and Windows shell.
8	HIGH	Confirm correct operation, including rollback and health check, with very large deployment files (on disk and via stdin).
9	HIGH	Validate import/export and cloud sync of deployment configurations, including error handling for upload/download failures and sync conflicts.
10	HIGH	Confirm that only supported file operations (import, export, edit) are allowed for YAML, JSON, TOML, and plain text files.
11	HIGH	Check accurate and secure log output in both verbose and non-verbose modes, ensuring no sensitive data (tokens/secrets) are logged.
12	HIGH	Verify crash reporting sends relevant error data, truncates PII, and prompts user appropriately per documentation.
13	HIGH	Test installation and basic operation of CLI Tool on Linux, macOS, and Windows, including via .tar.gz and all listed package managers.
14	HIGH	Validate Docker, Homebrew, apt, rpm, and Scoop installation images/packages for correct files, permissions, and path configurations.
15	HIGH	Check that CLI Tool launches and responds on all supported architectures (x86_64, ARM64, ARMv7) and OSes.
16	HIGH	Confirm primary commands function identically with all supported localization languages (English, German, French, Japanese).
17	MEDIUM	Validate shell integration: output formatting, colored output, and command piping work as documented on each platform.
18	MEDIUM	Test correct application of feature flags, ensuring that enabling/disabling features via flags takes effect immediately.
19	MEDIUM	Verify logging options: log file location, rotation, minimum/maximum verbosity, and error-only logs.
20	MEDIUM	Confirm that environment variable overrides for configs are honored and have correct precedence over config files.
21	MEDIUM	Confirm that interactive mode is available and appropriate in all localized UI languages.
22	MEDIUM	Validate plugin system: check detection, installation, and isolation of third-party/community plugins without impacting core CLI stability.
23	MEDIUM	Test third-party integrations (GitHub Actions, GitLab CI, CircleCI, Slack, PagerDuty, Datadog, HashiCorp Vault) for correct token management and data flow.
24	MEDIUM	Confirm that help, usage output, and error messages appear fully translated and contextually correct in all supported languages.
25	MEDIUM	Verify long-running session stability: health checks, streaming logs, and operation feedback for tasks exceeding 4 hours.
26	MEDIUM	Check performance requirements: rollout time, response to rapid command invocation, and behavior under load/large numbers of deployments.
27	LOW	Validate package manager uninstall process removes all CLI Tool files and environmental side-effects.
28	LOW	Confirm the behavior when invalid/unsupported config or data files are provided (e.g., XML file as config).
29	LOW	Verify all documentation URLs in CLI help reference and error output resolve correctly and point to https://docs.clitool.io .
30	LOW	Confirm the presence and clarity of usage instructions for portable builds and within Docker images.

Obr. 6.10: Checklist projektu CLITool — sekcia HIGH PRIORITY.